

Proyecto Integrador

Ingeniería de Sistemas

Evaluación y Mejora Experimental de un Algoritmo Evolutivo Multiobjetivo para la Descomposición de Monolitos en Microservicios

por Delfina Milagros Ferreri
Marzo 2026

dirigida por
Guillermo Rodríguez y
Ana Martínez Saucedo



UNICEN
Universidad Nacional del Centro
de la Provincia de Buenos Aires

Evaluación y Mejora Experimental de un Algoritmo Evolutivo Multiobjetivo para la Descomposición de Monolitos en Microservicios

– PROYECTO INTEGRADOR –
Carrera: Ingeniería de Sistemas

*Universidad Nacional del Centro de la Provincia de Buenos Aires,
Facultad de Ciencias Exactas.
(UNICEN, FCEX)*

15 de Marzo de 2026

Tandil, Buenos Aires, Argentina

Datos de la Alumna

Nombre: Delfina Milagros Ferreri
Mail: delfiferreri23@gmail.com
DNI: 44.377.571
N° de Legajo: 251147

Datos del Director

Nombre: Guillermo Rodríguez
Mail: exarodriguez@gmail.com

Datos de la Co-Directora

Nombre: Ana Martínez Saucedo
Mail: acmartinezsaucedo@gmail.com

Código fuente y experimentos reproducibles:
<https://github.com/equisdel/pps-pi.git>

Contenidos

Capítulo 1	3
1.1. Introducción	3
1.2. Problemática	3
1.3. Relevancia	4
1.4. Origen del Proyecto	5
1.5. Alcance	5
1.6. Equipo de Trabajo	6
1.7. Modalidad de Trabajo	7
1.8. Fases del Proyecto	7
Capítulo 2	9
2.1. Introducción	9
2.2. Tecnologías y Herramientas Empleadas	9
2.3. Formalización del Problema	10
2.4. Formalización del Enfoque	11
2.4.1. Representación de los Individuos	12
2.4.2. Función de Evaluación / Fitness	12
2.4.3. Población	12
2.4.4. Operador de Selección: Selección de Padres	12
2.4.5. Operador de Variación: Mecanismo de Mutación	13
2.4.6. Operador de Variación: Mecanismo de Cruzamiento	13
2.4.7. Operador de Selección: Selección de Sobrevivientes	13
2.4.8. Inicialización	13
2.4.9. Criterio de Convergencia	13
2.5. Casos de Estudio	14
2.6. Actividades Realizadas	14
Capítulo 3	16
3.1. Introducción	16
3.2. Análisis	17
3.2.1. Introducción de una Métrica de Calidad	17
3.2.2. Análisis de Sensibilidad	18
3.2.3. Estudio de Redundancia	22
3.3. Alteración	25
3.3.1. Implementación de Hypervolume (HV)	25
3.3.2. Modificación de la métrica NED	27
3.3.3. Reformulación del Individuo y Operadores Asociados	28
3.4. Evaluación	32
3.4.1. Impacto de la Nueva Representación	32
3.4.2. Nuevo Análisis de Sensibilidad	35
Capítulo 4	39
4.1. Conclusión	39
4.2. Trabajo Futuro	39
4.3. Reflexiones	39
Referencias	40

1.1. Introducción

La temática abordada en las Prácticas Profesionales Supervisadas (PPS) y el Proyecto Integrador (PI) se centra en la migración de sistemas monolíticos hacia arquitecturas basadas en microservicios.

En este proceso, una de las etapas más complejas consiste en descomponer el monolito en unidades más pequeñas y cohesionadas que puedan evolucionar como microservicios independientes. Dado que el espacio de soluciones crece superexponencialmente con el tamaño del sistema, la búsqueda de particiones adecuadas según múltiples objetivos constituye un problema perteneciente a la clase NP-Hard.

Para abordar este problema se adopta un enfoque de optimización multi-objetivo basado en computación evolutiva, donde cada posible descomposición del sistema es evaluada según distintos criterios arquitectónicos. En particular, se emplea un algoritmo evolutivo basado en NSGA-III con el fin de aproximar el frente de Pareto que representa las mejores soluciones de compromiso entre los objetivos en conflicto. Este trabajo retoma y extiende dicho enfoque, centrándose en medir y mejorar el desempeño del algoritmo existente mediante un análisis sistemático de su comportamiento.

Asimismo, se participó en el desarrollo de un *short-paper* enviado para evaluación a la conferencia *Genetic and Evolutionary Computation Conference (GECCO)*, una de las más relevantes en el área de la computación evolutiva. Al momento de escribir este informe, el artículo se encuentra en proceso de revisión. En caso de ser aceptado, podría ser presentado en la edición correspondiente al corriente año, a realizarse en San José, Costa Rica.

1.2. Problemática

La migración de código es hoy un problema concreto dentro de la arquitectura de software, que exige a las organizaciones una inversión significativa de tiempo y recursos. Empresas como Netflix y Amazon se han visto obligadas a migrar de una arquitectura a otra tras crecer rápidamente y encontrarse con los límites propios de los sistemas monolíticos.

Estas arquitecturas resultan prácticas en las primeras etapas del desarrollo, permitiendo avanzar rápido y simplificar el despliegue. Sin embargo, cuando los sistemas comienzan a crecer, esas mismas ventajas se vuelven obstáculos. La escalabilidad y la mantenibilidad se ven afectadas, ya que cada cambio implica recompilar grandes porciones del sistema. La flexibilidad disminuye y la robustez se vuelve frágil ante la constante evolución del producto. A esto se suma la complejidad que aparece cuando los equipos crecen y el código debe ser comprendido, modificado y coordinado por múltiples actores.

Frente a este escenario, las arquitecturas de microservicios se volvieron una alternativa popular al modelo monolítico. En lugar de concentrar toda la lógica en un único bloque, proponen distribuir las funcionalidades en servicios independientes que se despliegan por separado. Si bien su desarrollo y despliegue inicial suelen ser más complejos, esta estrategia se ve compensada por su mayor flexibilidad, aislamiento de fallos y la capacidad de escalar partes del sistema de forma independiente.

Fue a partir de mediados de la década de 2010 que el enfoque de microservicios comenzó a ganar una adopción significativa en la industria, impulsando numerosos procesos de migración en sistemas reales de gran escala. Entre los casos más citados se encuentra Netflix, que entre 2008 y 2015 migró desde una arquitectura monolítica en Java, con una base de datos centralizada, hacia un ecosistema distribuido de microservicios desplegados en la nube. La decisión estuvo motivada por la necesidad de eliminar puntos únicos de falla que afectaban la disponibilidad del sistema, evidenciada tras un incidente que dejó el servicio caído durante varios días.

Si bien la migración es una necesidad real que debe ser asistida por la tecnología, hasta la fecha no existe una herramienta capaz de automatizarla por completo. Un reciente *Systematic Mapping Study* (SMS) sobre migración a microservicios identifica la descomposición del monolito como la etapa más crítica: particionar el conjunto de componentes en subconjuntos que se convertirán en microservicios [1].

El desafío principal surge de la ausencia de un criterio único sobre qué significa que una arquitectura de microservicios sea "óptima". Los objetivos de diseño suelen entrar en conflicto entre sí: cohesión versus acoplamiento, granularidad versus costos de comunicación, entre muchos otros. De hecho, se han reportado alrededor de 63 criterios distintos para evaluar una descomposición [1], muchos de ellos contradictorios. Esto convierte al problema en un desafío que no admite una receta simple, sino que requiere balancear múltiples objetivos arquitectónicos.

Sumado a esto, el espacio de soluciones posibles para instancias de este problema suele ser enorme. En sistemas grandes, explorar todas las particiones manualmente es impracticable, y cualquier herramienta que intente cubrir este vacío debe lidiar con monolitos escritos en distintos lenguajes y estructurados de maneras poco convencionales, lo que convierte el análisis del código en un problema previo a la descomposición misma.

Es en este contexto que la descomposición de sistemas monolíticos en microservicios no solo aparece como una necesidad práctica, sino también como un problema formal de optimización arquitectónica con objetivos múltiples.

1.3. Relevancia

Como se señala en la sección anterior, la descomposición de sistemas monolíticos hacia arquitecturas más modulares constituye un problema recurrente y de alta relevancia tanto en la industria como en el ámbito académico. La literatura identifica esta descomposición como el problema más citado y central del proceso [1]. Si bien ha sido ampliamente estudiado, su aplicación en sistemas reales de gran escala continúa siendo compleja, lo que mantiene vigente la exploración de nuevos enfoques.

El SMS evidencia una marcada heterogeneidad en los enfoques propuestos y una ausencia de consenso respecto de procesos, técnicas, métricas y niveles de automatización. La mayoría de los trabajos se apoyan en estrategias manuales o semi-automatizadas, principalmente basadas en técnicas de inteligencia artificial, siendo el *clustering* el enfoque más utilizado.

Resulta llamativo que sólo una minoría de los trabajos formule explícitamente la descomposición del monolito como un problema de optimización combinatoria. En consecuencia, el uso de técnicas de computación evolutiva es limitado, representando los algoritmos genéticos apenas el 14% de los métodos no supervisados reportados. Esta baja adopción señala una brecha clara en la exploración sistemática del espacio de soluciones mediante enfoques evolutivos, que han demostrado ser metaheurísticas especialmente poderosas en este tipo de problemas.

Finalmente, se observa que la mayoría de los enfoques existentes no aborda el tratamiento de bases de datos independientes por microservicio, un principio fundamental en arquitecturas de microservicios. Si bien este aspecto introduce complejidad adicional, su incorporación dentro de una estrategia evolutiva resulta conceptualmente abordable y permite atacar una limitación clara de los métodos actuales, ampliando el alcance del modelo propuesto.

1.4. Origen del Proyecto

Esta iniciativa precede a mi participación. La incorporación al proyecto se produjo a partir del contacto con el director, Guillermo Rodríguez, quien propuso mi integración a esta línea de investigación ya que estaba vinculada con un área de mi interés, que es la computación evolutiva. Dicha línea había sido iniciada previamente por la doctoranda Ana Martínez Saucedo, con quien se estableció contacto para continuar el estudio del problema.

Como se mencionó antes, el trabajo previo abordaba la aplicación de técnicas de computación evolutiva al problema de optimización multiobjetivo que representa la descomposición de sistemas monolíticos en microservicios. En este contexto, el aporte principal de este proyecto consiste en retomar y extender dicho enfoque, teniendo como objetivo central la medición y mejora del desempeño del algoritmo existente.

Asimismo, se contó con el asesoramiento de Virginia Yannibelli, quien colaboró en la resolución de dudas conceptuales durante el proceso y tuvo una participación central en la elaboración del artículo presentado en el marco de la conferencia GECCO.

El organismo en el que se desarrollaron las Prácticas Profesionales Supervisadas (PPS) es la Facultad de Ciencias Exactas de la Universidad Nacional del Centro de la Provincia de Buenos Aires (UNICEN), institución en la que, con la entrega de este informe, culminaría mi formación de grado como Ingeniera de Sistemas.

1.5. Alcance

El trabajo desarrollado se centra en la etapa de descomposición de sistemas monolíticos hacia arquitecturas de microservicios. La propuesta toma como entrada un sistema monolítico implementado en Java, modelado mediante un grafo que representa las relaciones estructurales

entre clases y metadatos relevantes. Esta representación incluye principalmente referencias y llamados entre clases, información necesaria para el cálculo de métricas arquitectónicas como la cohesión y el acoplamiento.

A partir de dicha entrada, la definición del problema establece un conjunto de objetivos de optimización. En el caso de estudio considerado, se busca maximizar la cohesión interna de los módulos, minimizar el acoplamiento entre ellos y lograr una distribución balanceada de responsabilidades.

Sobre esta formulación se aplica un algoritmo evolutivo multiobjetivo basado en una versión del *Non-Dominated Sorting Genetic Algorithm* (NSGA-III) [2]. A lo largo de un número fijo de iteraciones, el algoritmo refina una población de particiones candidatas hasta aproximar el frente de Pareto del problema. La salida del proceso consiste en un conjunto de soluciones no dominadas que representan distintos compromisos entre los objetivos planteados, las cuales quedan disponibles para ser exploradas libremente como soporte al diseño arquitectónico.

El alcance del proyecto se limita a la descomposición lógica del monolito, por lo que no se aborda la migración automática del código, la generación de microservicios ejecutables ni su despliegue en entornos productivos.

1.6. Equipo de Trabajo

Guillermo Rodríguez

Rol principal: Director

Contribución: Guillermo Rodríguez participó como director del proyecto, brindando liderazgo y supervisión general a lo largo de sus etapas. Su aporte estuvo orientado a la definición del marco conceptual y metodológico, así como al acompañamiento en la toma de decisiones técnicas y organizativas. Asimismo, actuó como referente en aspectos administrativos y de gestión vinculados al desarrollo del trabajo.

Ana Martínez Saucedo

Rol principal: Co-Directora

Contribución: Ana Martínez Saucedo fue una de las principales impulsoras del proyecto, habiéndolo iniciado meses antes de mi incorporación. Mantuvo una interacción directa y continua durante el desarrollo, resultando clave para orientar el avance técnico y metodológico. Además, junto con Virginia Yannibelli, encabezó la redacción del trabajo presentado en GECCO.

Virginia Yannibelli

Rol principal: Consultora

Contribución: Virginia Yannibelli brindó asesoramiento especializado en el área de computación evolutiva, actuando como referente técnico en dicha temática. Su experiencia fue fundamental tanto para la resolución de dudas conceptuales como para la revisión y redacción del short paper, contribuyendo a fortalecer el marco teórico y metodológico del proyecto.

Delfina Milagros Ferreri (autora)

Rol principal: Analista / Programadora

Contribución: Fui responsable del análisis del problema, la revisión del enfoque previo, la actualización del algoritmo evolutivo, la ejecución de los experimentos y el análisis de los resultados obtenidos. Asimismo, participé en la parte técnica del artículo destinado a GECCO, integrando distintos componentes técnicos desarrollados durante el proyecto.

1.7. Modalidad de Trabajo

La modalidad de trabajo fue completamente remota. La comunicación con los integrantes del equipo se mantuvo de manera frecuente, principalmente a través de mensajería por Gmail y reuniones sincrónicas en Google Meet. La dinámica habitual consistía en encuentros destinados a acordar objetivos y definir una agenda de trabajo a corto plazo. A partir de estos acuerdos, se coordinaban nuevas reuniones en función del cumplimiento de deadlines, la presentación de avances o ante la necesidad de resolver dudas o solicitar asistencia.

El trabajo se desarrolló con un alto grado de autonomía, especialmente en las decisiones técnicas y de implementación, las cuales eran posteriormente discutidas y validadas en conjunto con la dirección del proyecto. Si bien no se siguió una metodología formal de gestión de proyectos, se trabajó con hitos definidos y fechas objetivo, priorizando avances incrementales y validaciones parciales.

1.8. Fases del Proyecto

El trabajo se desarrolló a lo largo de dos etapas consecutivas, las Prácticas Profesionales Supervisadas (PPS) y el Proyecto Integrador (PI), de acuerdo con el siguiente **cronograma**:

- Inicio de las PPS: 18 de agosto de 2025
- Finalización de las PPS: 15 de diciembre de 2025
- Inicio del PI: 16 de diciembre de 2025
- Envío de short-paper a *GECCO*: 26 de enero de 2026
- Finalización del PI: 15 de febrero de 2026

Durante este período, se mantuvieron reuniones sincrónicas según necesidad, aproximadamente cada dos o tres semanas, complementadas con comunicación asincrónica frecuente por correo electrónico.

Las actividades se organizaron en fases parcialmente solapadas, lo que permitió iteraciones y ajustes progresivos conforme avanzaba el trabajo. Con el objetivo de facilitar la lectura de los capítulos siguientes, el proyecto se estructura en las siguientes fases:

Fase 1. Lectura y preparación previa

(Agosto – Septiembre)

Etapla inicial orientada a la comprensión profunda del problema. Incluyó la lectura y análisis del SMS, así como la revisión de la teoría sobre computación evolutiva, aprendizaje sobre la optimización multi-objetivo y la inspección del enfoque general del proyecto. Esta fase fue clave para delimitar posibles líneas de aporte.

Fase 2. Análisis del modelo y generación de informes

(Septiembre – Octubre)

Una vez facilitado el repositorio original, se realizó un análisis detallado del código y del modelo propuesto. El objetivo fue comprender su funcionamiento, identificar supuestos, fortalezas y limitaciones, y evaluar el estado inicial del enfoque. A partir de este análisis se generaron informes técnicos que sirvieron de base para las decisiones posteriores.

Fase 3. Aportes directos al modelo y evaluación de mejoras *(Octubre – Diciembre)*

Con una comprensión sólida del modelo, se realizaron modificaciones y extensiones justificadas, orientadas a mejorar distintos aspectos de su desempeño. Los cambios fueron acompañados por evaluaciones comparativas, permitiendo medir su impacto sobre la performance del método.

Fase 4. Preparación del short-paper para GECCO y cierre *(Enero – Febrero)*

La etapa final consistió en una fase de trabajo colaborativo enfocado en la redacción y envío de un short-paper a la conferencia GECCO. Incluyó tareas de coordinación, reorganización del código en un nuevo repositorio, selección y síntesis de resultados relevantes, y adaptación del contenido a los estándares y tiempos propios de una convocatoria internacional.

La formalización del problema y del enfoque adoptado, así como las actividades realizadas, se precisan en el capítulo siguiente.

2.1. Introducción

El presente capítulo establece las bases técnicas del proyecto, formalizando tanto el problema abordado como el enfoque adoptado para su resolución. Mientras que el Capítulo 1 introduce el contexto y la motivación general, aquí se precisan los supuestos, definiciones y herramientas que sustentan el desarrollo experimental presentado posteriormente.

En particular, se expone una formulación explícita del problema, junto con el marco metodológico utilizado para abordarlo, con el objetivo de facilitar la comprensión del Capítulo 3, donde se detallan los experimentos realizados y los aportes concretos al modelo original.

La Sección 2.2 presenta las tecnologías y herramientas empleadas. La Sección 2.3 formaliza el problema, mientras que la Sección 2.4 describe el enfoque adoptado. Finalmente, la Sección 2.5 resume las actividades realizadas, estableciendo el vínculo directo con el capítulo siguiente.

2.2. Tecnologías y Herramientas Empleadas

Lenguajes y Entorno:

- **Python**: lenguaje principal utilizado para la implementación del modelo, la ejecución de experimentos y el análisis de resultados.
- **Java**: lenguaje principal de las instancias monolíticas de entrada admitidas por el modelo, base conceptual del proyecto.
- **Visual Studio Code**: editor principal de desarrollo.

Frameworks y Librerías:

- **DEAP**: framework evolutivo utilizado en la implementación original del modelo y extendido durante el desarrollo del proyecto.
- **pymoo**: soporte para análisis y experimentos secundarios con algoritmos multi-objetivo.
- **numpy, pandas, matplotlib**: manejo de datos, análisis estadístico y visualización.
- **SALib** (métodos de Sobol/Saltelli): análisis de sensibilidad global sobre los parámetros de entrada del modelo.

Herramientas de Trabajo Colaborativo:

- **GitHub**: control de versiones, organización del código, seguimiento, comunicación técnica.
- **Google Meet, Docs, Drive**: coordinación remota con la dirección, reuniones sincrónicas, intercambio de información y redacción de documentación.
- **Overleaf**: espacio de redacción colaborativa del *short-paper* para GECCO.
- **Asistentes de IA** (GitHub Copilot, ChatGPT, Claude, Edison Scientific): autocompletado de código, revisión de redacción técnica y mejora de claridad en la expresión escrita, siempre con validación manual del contenido.

2.3. Formalización del Problema

La problemática presentada en el Capítulo 1 puede interpretarse como una generalización del problema del empaquetado de contenedores (*bin-packing*), en tanto implica asignar elementos a contenedores, aunque en este caso el foco no se encuentra en la minimización del número de contenedores, sino en la optimización simultánea de múltiples criterios estructurales.

En el contexto específico de este trabajo, los elementos a asignar corresponden a clases de Java pertenecientes a un sistema monolítico, mientras que los contenedores se abstraen como conjuntos de clases que representan microservicios candidatos surgidos del proceso de descomposición.

Sea $N = \{1, \dots, n\}$ el conjunto de clases del sistema monolítico.

Una solución factible es una partición:

$$P = \{S_1, S_2, \dots, S_k\}$$

tal que:

- $S_i \subseteq N$
- $S_i \cap S_j = \emptyset$ si $i \neq j$
- $\bigcup_{i=1}^k S_i = N$

Cada subconjunto S_i representa un microservicio candidato.

A diferencia del *bin-packing* clásico, donde la calidad de una solución se mide a través de un único criterio, en este caso cada partición P se evalúa mediante un vector de objetivos:

$$F(P) = (f_1(P), f_2(P), f_3(P), f_4(P))$$

donde cada componente corresponde a una métrica estructural que captura propiedades deseables en arquitecturas de microservicios.

De acuerdo con [3], la descomposición del monolito se modela como un problema de optimización multiobjetivo con cuatro objetivos principales:

Nombre	Descripción	Descripción Matemática
Non-Extreme Distribution (NED)	Mide la proporción de microservicios cuyo tamaño resulta irregular (menos de 5 clases o más de 20). Evalúa la distribución de tamaño de los servicios, favoreciendo particiones balanceadas.	Minimizar: $f_1 = NED = 1 - \frac{ \{M_i \mid 5 < M_i < 20\} }{k}$
Structural Modularity (SM)	Recompensa la alta cohesión* interna y penaliza el acoplamiento** externo.	Maximizar: $f_2 = SM = \frac{1}{k} \sum_{i=1}^k \frac{ E_c(M_i, M_i) }{ M_i ^2} - \frac{1}{\frac{k(k-1)}{2}} \sum_{i \neq j} \frac{ E_c(M_i, M_j) }{2 M_i M_j }$

Inter-Call Percentage (ICP)	Mide la proporción de llamadas entre microservicios respecto del total de llamadas del sistema. Captura el acoplamiento dinámico basado en invocaciones entre clases.	Minimizar: $f_3 = ICP = \frac{\sum_{i=1}^k \sum_{j \neq i}^k icp(M_i, M_j)}{\sum_{i=1}^k \sum_{j=i}^k icp(M_i, M_j)}$ where: $icp(M_i, M_j) = \sum_{c_k \in M_i} \sum_{c_l \in M_j} (\log(calls(c_k, c_l)) + 1)$
Interface Number (IN)	Cuantifica la cantidad de dependencias estructurales entre microservicios distintos. Refuerza la minimización del acoplamiento estructural.	Minimizar: $f_4 = IN = \frac{1}{k} \sum_{i=1}^k \sum_{j \neq i}^k E_c(M_i, M_j) $

* **Cohesión:** alta densidad de dependencias dentro de un mismo microservicio.

** **Acoplamiento:** dependencias entre distintos microservicios.

Dado que estos objetivos pueden resultar conflictivos entre sí (por ejemplo, aumentar cohesión puede afectar la distribución de tamaños), no existe una única solución óptima global. Se adopta entonces el concepto de dominancia de Pareto:

Una solución P_1 domina a P_2 si:

- No es peor en ningún objetivo, y
- Es estrictamente mejor en al menos uno.

El conjunto de soluciones no dominadas conforma el frente de Pareto, donde cada solución representa un compromiso distinto entre cohesión, acoplamiento y balance estructural.

Este encuadre habilita la formulación del problema como uno de optimización combinatoria multiobjetivo. Computacionalmente, el problema presenta complejidad NP-Hard: el espacio de soluciones crece superexponencialmente en función de N, haciendo impracticable la exploración exhaustiva. Sin algoritmos que garanticen optimalidad en tiempo polinómico, se requieren estrategias metaheurísticas para abordarlo.

Esto justifica el enfoque evolutivo desarrollado a continuación.

2.4. Formalización del Enfoque

En su versión original, el enfoque adoptado en el proyecto sigue la estructura clásica de un algoritmo evolutivo descrito en [3, Sec. 3.2]. La implementación se realizó utilizando el framework DEAP [4], lo que determina y, en algunos casos, condiciona ciertos aspectos prácticos de su configuración.

El algoritmo recibe como entrada un grafo de dependencias previamente computado, que modela las relaciones principales entre clases para una instancia monolítica, y devuelve como salida una aproximación del frente de Pareto ideal, es decir, un conjunto de particiones no dominadas.

El desempeño del enfoque se evalúa en términos de la calidad y estabilidad de los frentes de Pareto obtenidos, aspecto que se desarrolla en la Sección 3.3.1.

Un algoritmo evolutivo se compone de una representación, una función de evaluación, operadores de variación y selección, una estrategia poblacional y un criterio de parada. En las secciones siguientes se presentan dichos componentes fundamentales, mientras que las modificaciones introducidas sobre esta base se describen más adelante.

2.4.1. Representación de los Individuos

Sea N el número de clases del monolito. Cada individuo (o partición) se representa como un vector:

$$P = [m_1, m_2, \dots, m_N]$$

donde $m_i \in [0, N - 1]$ indica la etiqueta del microservicio al cual se asigna la clase i .

Durante la evaluación, esta representación se transforma en un diccionario de la forma:

$$P' = \{k: \{i \in \{0, \dots, N - 1\} \mid m_i = k\}\}$$

lo que permite agrupar las clases pertenecientes a un mismo microservicio y simplificar los cálculos de las métricas asociadas a la función de evaluación (Sección 2.4.2).

Si bien esta representación resulta simple y computacionalmente eficiente, induce una redundancia estructural asociada a la permutación arbitraria de etiquetas que expande artificialmente el espacio de búsqueda. Este fenómeno constituye uno de los principales hallazgos del proyecto y será analizado en la Sección 3.2.3, con alteraciones detalladas en la Sección 3.3.3 y evaluadas en la Sección 3.4.1.

2.4.2. Función de Evaluación / Fitness

La evaluación de cada individuo se realiza aplicando la función multiobjetivo $F(P)$ definida en la Sección 2.3. Las métricas estructurales (SM , ICP e IN) se calculan sobre el grafo de dependencias que toma como entrada el algoritmo. En contraste, la métrica NED depende únicamente de la distribución de clases entre microservicios y no requiere información adicional del grafo.

2.4.3. Población

El algoritmo mantiene un tamaño de población fijo por ejecución (μ). Dicho tamaño constituye un hiperparámetro configurable dentro del framework, aunque en la configuración original se emplea un valor constante.

2.4.4. Operador de Selección: Selección de Padres

No se emplea un operador de selección explícito para la elección de padres. La generación de descendencia se realiza mediante muestreo aleatorio de individuos de la población actual, seguido de la aplicación de operadores de variación (mutación o cruzamiento) según probabilidades predefinidas.

Las probabilidades de mutación $p_{mut} \in [0, 1]$ y de cruzamiento $p_{cx} \in [0, 1]$ constituyen hiperparámetros del algoritmo. En la configuración adoptada, ambas se definen de manera complementaria:

$$p_{cx} = 1 - p_{mut}$$

2.4.5. Operador de Variación: Mecanismo de Mutación

El operador de mutación actúa sobre la representación vectorial del individuo. En cada aplicación, se selecciona uniformemente una posición $i \in \{1, \dots, N\}$, correspondiente a una clase del monolito, y se reasigna su etiqueta de microservicio mediante una selección aleatoria uniforme en el rango permitido. Formalmente, dado un individuo $P = \{m_1, \dots, m_N\}$ la mutación genera un nuevo individuo P' tal que:

$$i \sim U(0, N - 1); \quad m_i' \sim U(0, N - 1)$$

2.4.6. Operador de Variación: Mecanismo de Cruzamiento

El operador de cruzamiento empleado es el cruzamiento de un punto (one-point crossover). Dados dos individuos parentales P_1 y P_2 , se selecciona aleatoriamente un punto de corte $k \in \{1, \dots, N - 1\}$ y se intercambian los segmentos posteriores a dicho punto, generando dos descendientes. De los dos descendientes generados, uno es incorporado a la descendencia final, mientras que el otro se descarta, de acuerdo con el esquema implementado en DEAP.

2.4.7. Operador de Selección: Selección de Sobrevivientes

La población de la generación siguiente se obtiene mediante un esquema $(\mu + \lambda)$, en el cual se seleccionan μ individuos a partir de la unión entre la población parental y los λ descendientes generados. La selección ambiental se realiza utilizando NSGA-III, basado en ordenamiento por frentes no dominados y niching mediante puntos de referencia. Al igual que μ , λ es un hiperparámetro configurable.

2.4.8. Inicialización

La inicialización de la población es aleatoria.

2.4.9. Criterio de Convergencia

El algoritmo emplea un criterio de parada fijo, deteniéndose luego de 100 generaciones ejecutadas.

Para recapitular, se identifican como hiperparámetros principales del enfoque las variables μ (tamaño de la población), λ (cantidad de offsprings por generación) y las probabilidades complementarias p_{mut} y p_{cx} .

Este enfoque se probó y se evaluó con distintas instancias conocidas de monolitos, de tamaños variados, que se detallan en la sección siguiente.

2.5. Casos de Estudio

Los casos de estudio involucrados en el proyecto fueron:

Instancia	JPetStore	AcmeAir	CargoTracker
Descripción	Aplicación monolítica de e-commerce utilizada como benchmark académico para estudios de arquitectura y migración	Sistema de reservas aéreas que gestiona vuelos, clientes y operaciones de booking. Desarrollada por IBM para benchmarking	Sistema de gestión logística orientado al seguimiento y transporte de cargas, basado en Domain-Driven Design (DDD) como caso de estudio
Tamaño (N)	24	32	54

El principal caso analizado fue JPetStore [5], mientras que CargoTracker [6] se utilizó para evaluar la estabilidad del enfoque en una instancia de mayor tamaño. Posteriormente, durante el proceso de preparación del trabajo para *GECCO*, AcmeAir [7] reemplazó parcialmente a CargoTracker debido a la existencia de estudios previos más consolidados sobre dicha instancia.

Estas instancias fueron seleccionadas por constituir benchmarks recurrentes en estudios de descomposición de monolitos y migración a microservicios. Su uso facilita la comparabilidad con trabajos previos y permite contextualizar los resultados obtenidos dentro del estado del arte.

Si bien estas aplicaciones son representativas en la literatura, su tamaño es relativamente pequeño en comparación con sistemas industriales reales, que suelen involucrar cientos de clases. En consecuencia, los resultados obtenidos deben considerarse dentro de ese alcance.

2.6. Actividades Realizadas

A lo largo del proyecto, mi rol evolucionó desde tareas predominantemente analíticas en las etapas iniciales, centradas en la comprensión formal del problema y del estado del arte, hacia actividades de desarrollo e implementación. En la etapa final, el trabajo retomó un carácter analítico, enfocado en la interpretación, síntesis y comunicación de los resultados obtenidos.

Dicho esto, las actividades desarrolladas sobre el enfoque original pueden agruparse en tres categorías principales:

1. **Análisis:** estudio conceptual y experimental del comportamiento del algoritmo, orientado a caracterizar su desempeño, estabilidad y limitaciones estructurales.
2. **Alteración:** modificaciones introducidas sobre componentes del enfoque original, incluyendo métricas, representación y operadores.

3. **Evaluación:** validación cuantitativa de las modificaciones propuestas mediante métricas de desempeño y análisis comparativos.

Si bien se presentan agrupadas bajo estas tres categorías, las actividades no fueron lineales sino iterativas, retroalimentándose a lo largo del proyecto.

En lo que sigue se detallan los aportes específicos realizados, omitiendo aquellas actividades intrínsecas a todo proyecto de investigación y desarrollo (revisión bibliográfica, definición de objetivos, descomposición del problema, elaboración de informes internos, entre otras).

2.6.1. Análisis del enfoque original

- Determinación de métricas de calidad para el frente de Pareto.
- Exploración del espacio de soluciones y estudio de los rangos de la función de fitness para cada objetivo en las instancias consideradas.
- Análisis de sensibilidad, incertidumbre y robustez mediante muestreo Saltelli para caracterizar la influencia de los hiperparámetros sobre el desempeño del algoritmo.
- Identificación de redundancia estructural en la representación de los individuos, asociada a la permutación arbitraria de etiquetas.

2.6.2. Alteraciones Introducidas

- Corrección y reformulación de la métrica NED.
- Implementación de métricas de desempeño para evaluar cuantitativamente la calidad de los frentes obtenidos, centrada en Hypervolume (HV).
- Rediseño de la representación de los individuos, así como de los operadores de variación asociados, para optimizar la dinámica de exploración del algoritmo.
- Refactoring completo del código, así como otras correcciones enfocadas a incrementar la eficiencia y/o la reproducibilidad de los estudios.

2.6.3. Evaluación de las modificaciones

- Comparación experimental entre el enfoque original y el modificado.
- Análisis cuantitativo de los frentes de Pareto resultantes.
- Evaluación del impacto de los cambios en términos de calidad, estabilidad y diversidad de soluciones.

2.6.4. Difusión

- Consolidación de los resultados.
- Participación en la elaboración de material para su presentación en GECCO.

Las actividades previamente enumeradas no fueron independientes entre sí, sino que conformaron un proceso iterativo de diagnóstico, intervención y validación. El capítulo siguiente expone este proceso en detalle, formalizando cada análisis realizado, justificando las decisiones de rediseño adoptadas y presentando la evidencia experimental que sustenta las conclusiones obtenidas.

3.1. Introducción

Este capítulo presenta los principales avances obtenidos durante el desarrollo del enfoque propuesto, organizados en tres líneas de trabajo complementarias: análisis del enfoque original, estudio de posibles modificaciones y evaluación experimental de dichas alternativas.

En primer lugar, se identificó la necesidad de incorporar una métrica cuantitativa de calidad que permitiera evaluar de manera objetiva el desempeño del algoritmo evolutivo y comparar distintas configuraciones del enfoque. Para ello se adoptó el indicador Hypervolume (HV), ampliamente utilizado en la literatura de optimización multi-objetivo para evaluar simultáneamente la convergencia y la diversidad de los frentes de Pareto aproximados.

Con el fin de aplicar esta métrica de manera adecuada, se realizó posteriormente una exploración del espacio de soluciones, orientada a comprender el comportamiento de las funciones objetivo y estimar los rangos de valores alcanzables para cada instancia. Este análisis, basado en muestreo Monte Carlo, permitió obtener información empírica sobre la estructura del espacio objetivo y facilitó la definición del punto de referencia necesario para el cálculo del HV. Además, reveló una inconsistencia en la métrica NED, que tendía a permanecer constante, por lo que fue reajustada para ser proporcional al tamaño de la instancia.

Una vez definida la métrica de evaluación, se llevó a cabo un análisis de sensibilidad del algoritmo, con el objetivo de estudiar la influencia de sus principales hiperparámetros sobre la calidad de las soluciones obtenidas. Este análisis fue solicitado explícitamente durante la revisión del trabajo presentado en *GECCO 2025* y constituye un paso fundamental para evaluar la robustez del enfoque y ajustar sus parámetros de manera informada. Para ello se utilizó un diseño experimental basado en muestreo de Sobol/Saltelli.

Posteriormente, a partir del análisis del comportamiento del algoritmo, se exploraron posibles modificaciones en la representación de los individuos y en los operadores de variación, con el objetivo de mejorar la eficiencia del proceso evolutivo.

Finalmente, se realiza una evaluación experimental del enfoque resultante, analizando la calidad de los frentes de Pareto obtenidos y la dinámica de convergencia del algoritmo. Además, se repite el análisis de sensibilidad para estudiar el comportamiento del método bajo distintas configuraciones. El capítulo concluye con una síntesis de los resultados obtenidos y una discusión de posibles líneas de trabajo futuras.

Cabe destacar que estas líneas de trabajo no se desarrollaron de manera estrictamente secuencial, sino de forma iterativa, retroalimentándose entre sí a lo largo del proceso de investigación. Por esta razón, si bien el capítulo se organiza siguiendo una estructura lógica de presentación, cada sección puede leerse de forma relativamente independiente según el aspecto del enfoque que resulte de mayor interés.

3.2. Análisis

3.2.1. Introducción de una Métrica de Calidad

Motivación

Para mejorar el enfoque original, resulta necesario contar con mecanismos objetivos que permitan cuantificar la calidad de las soluciones obtenidas.

Hasta el momento, el algoritmo produce como salida un frente de Pareto, cuya evaluación se ha realizado principalmente mediante representaciones gráficas y comparaciones con resultados reportados en la literatura por otros investigadores que emplean enfoques no evolutivos. Este tipo de análisis fue suficiente para la primera presentación del trabajo en GECCO. Sin embargo, dicho enfoque resulta limitado cuando se busca comparar distintas configuraciones del propio algoritmo.

En particular, la necesidad de realizar análisis de sensibilidad y estudios comparativos entre variantes del método hace imprescindible disponer de métricas cuantitativas que permitan evaluar de forma consistente la calidad de los frentes de Pareto generados.

En este contexto surge la siguiente pregunta:

- ¿Cómo cuantificar de manera fiable la calidad de un frente de Pareto?

Definición

En la optimización multi-objetivo mediante algoritmos evolutivos, el desempeño de un algoritmo se evalúa mediante indicadores de calidad, los cuales cuantifican propiedades del conjunto de soluciones aproximado al frente de Pareto, tales como convergencia, diversidad y cobertura.

La literatura reporta una gran variedad de indicadores de calidad propuestos para este propósito. Sin embargo, varios estudios de revisión muestran que el uso práctico se concentra en un conjunto relativamente pequeño de métricas ampliamente adoptadas, entre las que destacan Hypervolume (HV), Generational Distance (GD), Inverted Generational Distance (IGD), el indicador ϵ (epsilon) y métricas de diversidad como Spread (Δ) [8][9].

Entre estos indicadores, el Hypervolume (HV) es frecuentemente reportado como uno de los más utilizados en estudios comparativos de algoritmos evolutivos multi-objetivo, ya que permite evaluar simultáneamente convergencia y diversidad del conjunto de soluciones aproximado [8].

Por su amplia adopción en la literatura, su carácter unario, y el hecho de que solo requiere la definición de un punto de referencia, en contraste con otros indicadores que requieren una aproximación del frente de Pareto ideal, la métrica central utilizada en este estudio es el Hypervolume (HV).

Interpretación del Hypervolume (HV)

El HV mide el volumen del espacio de objetivos que es dominado por el frente de Pareto aproximado y delimitado por un punto de referencia previamente definido.

Intuitivamente, cada solución del frente domina una región del espacio de objetivos. El HV corresponde al volumen total cubierto por la unión de dichas regiones dominadas. En consecuencia, valores mayores de HV indican frentes que se encuentran simultáneamente más cercanos al frente de Pareto óptimo y mejor distribuidos a lo largo del espacio de soluciones.

Para su cálculo es necesario definir un punto de referencia, que representa una solución hipotética peor que cualquier solución del frente en todos los objetivos. El HV se obtiene entonces como el volumen del espacio de objetivos dominado por el frente y limitado por dicho punto.

En este trabajo, el HV se emplea como indicador principal para evaluar el desempeño de las distintas configuraciones del algoritmo. Los detalles de su implementación se discuten en la Sección 3.3.1, incluido el análisis del espacio de soluciones que permitió determinar el punto de referencia.

3.2.2. Análisis de Sensibilidad

Motivación

Con el objetivo de comprender cómo los hiperparámetros del algoritmo influyen en la calidad de las soluciones obtenidas, se realizó un análisis de sensibilidad global sobre el algoritmo evolutivo. Este tipo de análisis permite cuantificar la contribución de cada parámetro a la variabilidad de la salida del modelo y detectar posibles interacciones entre ellos.

En este contexto surge la siguiente pregunta:

- ¿Cuál es la influencia relativa de los hiperparámetros del algoritmo evolutivo sobre la calidad de las soluciones obtenidas, medida a través del indicador Hypervolume (HV)?

Metodología

Para este estudio se consideraron tres hiperparámetros principales del algoritmo: el tamaño poblacional (μ), la relación entre descendientes y población (λ/μ) y la probabilidad de mutación (p_{mut}). La probabilidad de cruzamiento se definió como el complemento de la probabilidad de mutación ($1 - p_{mut}$), siguiendo el esquema de operadores utilizado en el framework DEAP [4].

Los rangos seleccionados para el muestreo Saltelli, que genera muestras cuasialeatorias en el espacio de parámetros, se definieron con el objetivo de cubrir un espectro amplio de configuraciones posibles del algoritmo. En particular, se exploró un tamaño poblacional $\mu \in [50, 1000]$, una relación entre descendientes y población $\lambda/\mu \in [1.0, 3.0]$, y una probabilidad de mutación $p_{mut} \in [0.0, 1.0]$. Estos intervalos permiten considerar desde configuraciones con poblaciones relativamente pequeñas hasta escenarios con poblaciones grandes y mayor producción de descendientes.

La exploración del espacio de hiperparámetros se realizó mediante muestreo Saltelli utilizando la librería SALib [10]. Se utilizó un tamaño base $N = 1024$. Considerando los tres parámetros de entrada y deshabilitando el cálculo de índices de segundo orden, el muestreo Saltelli generó un total de 5120 configuraciones. Cada configuración fue utilizada para ejecutar una corrida independiente del algoritmo evolutivo sobre la instancia JPetStore, registrando como salida el valor final del

indicador Hypervolume (HV) correspondiente al frente de Pareto obtenido, calculado respecto al punto de referencia fijo registrado en la Sección 3.3.1 para dicha instancia.

Los valores obtenidos de HV se utilizaron para realizar un análisis de sensibilidad de Sobol. Los índices de Sobol permiten distinguir entre el efecto directo de cada parámetro sobre la variable de salida y el efecto total que incluye interacciones con otros parámetros. Se calcularon los índices de primer orden (S1), que capturan el efecto directo de cada parámetro, y los índices de orden total (ST), que incluyen además las interacciones con otros parámetros.

El cálculo de índices de segundo orden fue deshabilitado, ya que los índices totales (ST) capturan el efecto agregado de dichas interacciones. Esta decisión permitió reducir el costo computacional del análisis sin afectar la identificación de los hiperparámetros más influyentes.

Resultados

Los índices de Sobol obtenidos para cada hiperparámetro se presentan en la Tabla 3.2.1.

Parámetro	S1	ST	IC(S1)	IC(ST)	ST – S1
μ	0.3617	0.5985	0.0693	0.0565	0.2368
p_mut	0.2358	0.5016	0.0693	0.0543	0.2658
λ/μ	0.1665	0.4071	0.0547	0.0464	0.2406

Tabla 3.2.1: Índices de sensibilidad de Sobol para los hiperparámetros del algoritmo evolutivo, calculados sobre el indicador Hypervolume (HV). Se reportan los índices de primer orden (S1), los índices totales (ST) y los intervalos de confianza (IC) asociados.

Interpretación

Los resultados obtenidos indican que el tamaño poblacional (μ) es el parámetro con mayor influencia sobre la variabilidad del HV. Su índice de primer orden alcanza un valor de S1 = 0.3617, lo que implica que una proporción importante de la variabilidad observada en el indicador puede explicarse por cambios en el tamaño poblacional. Al considerar también las interacciones con otros parámetros, su influencia total asciende a ST = 0.5985.

En segundo lugar aparece la probabilidad de mutación (p_mut), con S1 = 0.2358 y ST = 0.5016. Esto indica que la probabilidad de mutación tiene una influencia directa moderada sobre el desempeño del algoritmo, que aumenta al considerar sus interacciones con otros hiperparámetros.

Por su parte, la relación entre descendientes y población (λ/μ) presenta el menor efecto directo (S1 = 0.1665), aunque su índice total (ST = 0.4071) sugiere que también participa en interacciones relevantes con los demás parámetros del algoritmo.

La diferencia entre los índices totales y de primer orden permite estimar la magnitud de dichas interacciones. En este sentido, todos los parámetros presentan contribuciones de interacción relativamente similares, lo que indica que el comportamiento del algoritmo no depende únicamente de efectos aislados, sino también de la combinación entre hiperparámetros.

Cabe aclarar que dado el carácter estocástico de los algoritmos evolutivos, la evaluación de cada configuración de hiperparámetros presenta variabilidad natural. Esto se observa claramente en las figuras animadas de la subsección siguiente (Figura 3.2.1). Un desafío adicional es que el framework DEAP no garantiza una reproducibilidad completamente determinística, aun cuando se fijan semillas aleatorias, por lo que parte de la variabilidad observada en el HV puede atribuirse al ruido.

Finalmente, los intervalos de confianza asociados a cada índice muestran que la influencia de μ se mantiene consistente incluso considerando la incertidumbre de la estimación. En conjunto, estos resultados sugieren que el tamaño poblacional constituye el principal determinante del desempeño del algoritmo en términos de Hypervolume, mientras que la probabilidad de mutación y la proporción de descendientes ejercen una influencia secundaria pero no despreciable.

Otros Hallazgos

Más allá del análisis de sensibilidad, la información obtenida a partir de las 5120 ejecuciones, registrando no solo el valor final de HV sino también los frentes de Pareto resultantes, constituye una base de datos que permite calcular métricas adicionales y realizar análisis complementarios.

Entre ellos, se exploró qué configuraciones de hiperparámetros tienden a producir mejores resultados. Para ello se visualizaron las configuraciones correspondientes al mejor X % y al peor X % de ejecuciones mediante gráficos tridimensionales, donde los ejes representan los tres hiperparámetros considerados y el color indica la calidad del HV alcanzado (verde para valores altos y rojo para valores bajos).

La animación pone de manifiesto que los mejores resultados se obtienen con valores altos de μ , es decir, poblaciones grandes. Asimismo, resulta conveniente generar una mayor cantidad de individuos por generación (λ alto en proporción a μ). También se favorecen configuraciones con mayor probabilidad de mutación, mientras que aquellas basadas predominantemente en cruzamiento tienden a producir resultados de menor calidad.

A modo ilustrativo, en la Tabla 3.2.2, se presentan las estadísticas descriptivas correspondientes al 1% superior e inferior de ejecuciones según su valor de HV.

	top 1%					bottom 1%				
	mu	lambda	p_mut	p_cx	HV	mu	lambda	p_mut	p_cx	HV
count	52	52	52	52	52	52	52	52	52	52
mean	794.75	2126.06	0.7141	0.2858	4.0454	132.30	227.88	0.0812	0.9187	1.8516
std	130.47	371.86	0.1567	0.1567	0.0189	85.32	173.68	0.1871	0.1871	0.2538
cv	0.1641	0.1749	0.2194	0.5482	0.0046	0.6448	0.7621	2.3041	0.2036	0.13706
min	430.00	1214.00	0.2653	0.1691	4.0219	53.00	60.00	0.0003	0.0050	1.1261
max	972.00	2843.00	0.9592	0.7346	4.1060	375.00	829.00	0.9949	0.9997	2.1217

Tabla 3.2.2: Estadísticas descriptivas de los hiperparámetros y del indicador Hypervolume (HV) para las configuraciones correspondientes al 1% superior y al 1% inferior de ejecuciones. Se reportan el número de observaciones (count), media (mean), desvío estándar (std), coeficiente de variación (cv), y los valores mínimo y máximo observados.

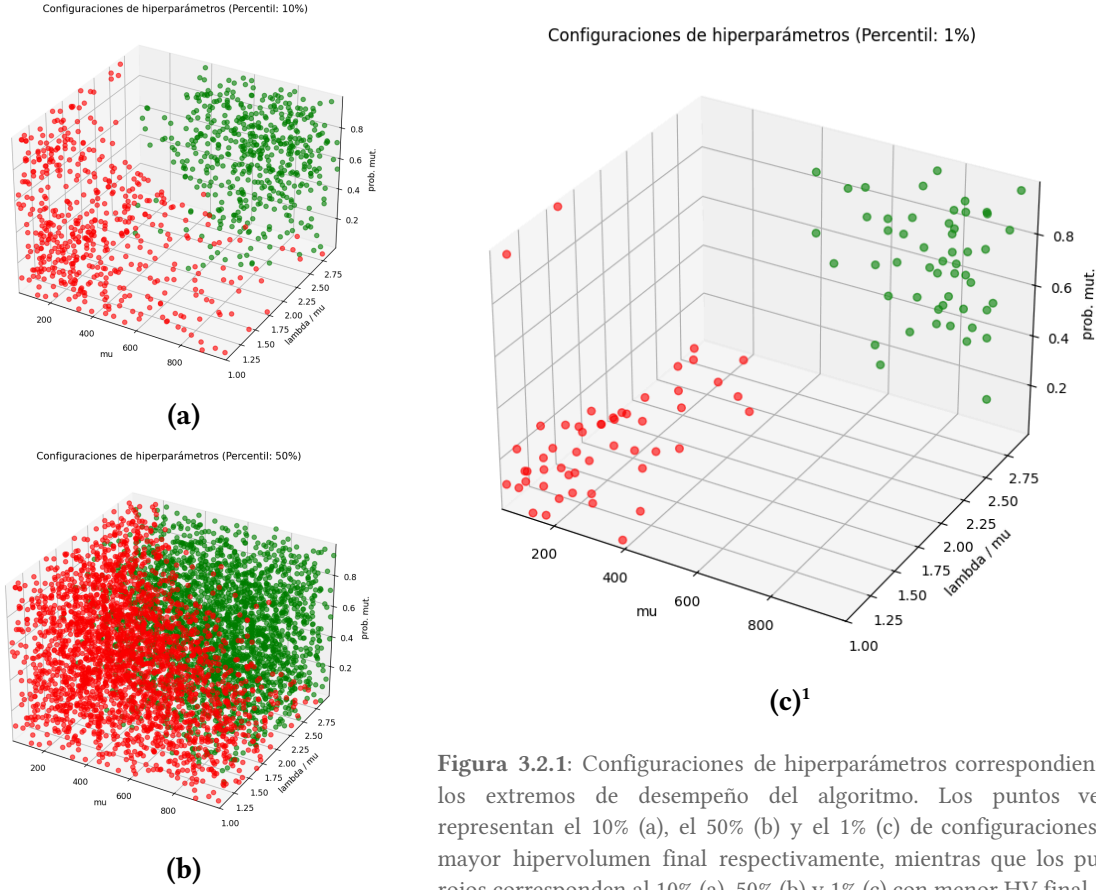


Figura 3.2.1: Configuraciones de hiperparámetros correspondientes a los extremos de desempeño del algoritmo. Los puntos verdes representan el 10% (a), el 50% (b) y el 1% (c) de configuraciones con mayor hipervolumen final respectivamente, mientras que los puntos rojos corresponden al 10% (a), 50% (b) y 1% (c) con menor HV final.

El contraste entre ambos grupos resulta particularmente marcado. Las ejecuciones dentro del 1 % superior utilizan poblaciones aproximadamente seis veces mayores que las del 1 % inferior ($\lambda \approx 795$ frente a $\lambda \approx 132$), mientras que el número de descendientes generados por generación es aproximadamente nueve veces mayor ($\lambda \approx 2126$ frente a $\lambda \approx 228$). Estos resultados sugieren que mantener poblaciones amplias y una elevada tasa de generación de nuevos individuos es fundamental para explorar adecuadamente el espacio de soluciones.

En cuanto a los operadores evolutivos, las configuraciones de mejor desempeño se caracterizan por ser predominantemente impulsadas por mutación ($p_{\text{mut}} \approx 0.71$), mientras que las configuraciones de peor desempeño dependen casi exclusivamente del cruzamiento ($p_{\text{cx}} \approx 0.92$). Esto sugiere que la mutación cumple un papel dominante en la generación de variación útil dentro del espacio de soluciones, mientras que el cruzamiento por sí solo resulta insuficiente para producir frentes de Pareto de alta calidad.

La diferencia en la calidad de las soluciones obtenidas también se refleja directamente en el valor de HV. Las ejecuciones del grupo superior alcanzan valores promedio de $\text{HV} \approx 4.05$, más del doble que las del grupo inferior ($\text{HV} \approx 1.85$), lo que indica que la elección de hiperparámetros tiene un impacto sustancial en el desempeño del algoritmo.

Finalmente, se observa una diferencia notable en la estabilidad de ambos grupos. Las configuraciones de alto desempeño presentan una variabilidad extremadamente baja en el HV ($\text{CV} \approx$

¹ La animación que muestra la oscilación del 1% al 50% de los extremos de HV está disponible en: [original.gif](#)

0.004), mientras que las configuraciones de peor desempeño muestran una variabilidad considerablemente mayor ($CV \approx 0.254$). Esto sugiere que las regiones del espacio de hiperparámetros asociadas a buenos resultados son relativamente estables, mientras que las regiones de bajo desempeño tienden a producir resultados más erráticos.

La unión de los frentes de Pareto obtenidos en todas las ejecuciones también se utilizó como aproximación a un frente de referencia, permitiendo calcular métricas adicionales de calidad. Asimismo, a partir de los frentes almacenados fue posible calcular la redundancia, proceso que se detalla en la sección siguiente y que motivó el cambio principal documentado en este proyecto.

3.2.3. Estudio de Redundancia

Motivación

A partir del análisis del espacio de soluciones realizado para determinar el punto de referencia de HV (Sección 3.2.1) y del estudio exhaustivo del comportamiento del enfoque original mediante múltiples iteraciones (Sección 3.2.2), surgió la sospecha de que la representación de los individuos podría estar introduciendo ineficiencias en la exploración del espacio de búsqueda. Puntualmente el análisis de sensibilidad reveló la ineficacia del operador de cruzamiento en contraste con el de mutación, algo inusual en este tipo de algoritmos.

Definición

Una observación preliminar motivó un análisis más profundo: el caso extremo en el que todas las clases se asignan a un único microservicio. Según la representación detallada en la Sección 2.4.1, esta solución podría codificarse como $[0,0,\dots,0]$, pero también como $[1,1,\dots,1]$, $[2,2,\dots,2]$, ..., $[N-1,N-1,\dots,N-1]$. Todas estas codificaciones representan exactamente la misma partición conceptual. Sin embargo, el algoritmo las considera individuos distintos.

Este fenómeno se conoce como redundancia por permutación de etiquetas. A continuación se presenta un ejemplo puntual del mismo, observado en uno de los frentes de Pareto obtenidos para la instancia JPetStore:

Canonical partition: [[0],[1,2,3,4,5,6,8,9,10,11,13,19],[7],[12,17,20,21,22,23],[14],[15],[16],[18]]
Size: 8
Count: 10
Equivalent representations in Pareto Front: [3 18 18 18 18 18 18 4 18 18 18 18 5 18 11 23 10 5 14 18 5 5 5 5] [22 18 18 18 18 18 18 12 18 18 18 18 5 18 11 23 10 5 16 18 5 5 5 5] [0 18 18 18 18 18 18 17 18 18 18 18 5 18 11 23 10 5 14 18 5 5 5 5] [16 18 18 18 18 18 18 4 18 18 18 18 5 18 11 23 20 5 1 18 5 5 5 5] [4 18 18 18 18 18 18 17 18 18 18 18 5 18 23 8 19 5 14 18 5 5 5 5] [13 18 18 18 18 18 18 17 18 18 18 18 5 18 11 23 10 5 19 18 5 5 5 5] [4 18 18 18 18 18 18 3 18 18 18 18 5 18 21 23 10 5 13 18 5 5 5 5] [3 18 18 18 18 18 18 17 18 18 18 18 5 18 11 23 20 5 1 18 5 5 5 5] [0 18 18 18 18 18 18 17 18 18 18 18 5 18 11 23 4 5 19 18 5 5 5 5]

[3 18 18 18 18 18 18 4 18 18 18 18 5 18 11 23 20 5 1 18 5 5 5 5]
fitness: (NED: 0.75, SM: 0.7259, ICP: 0.3695, IN: 1.5)

Esto resulta potencialmente problemático porque la representación adoptada induce una relación no biyectiva entre genotipo y fenotipo. En metaheurísticas poblacionales, la representación define la estructura efectiva del espacio sobre el cual operan los mecanismos de variación, por lo que no constituye un aspecto meramente técnico sino estructural.

La redundancia no es necesariamente negativa. En algoritmos evolutivos es habitual la presencia de individuos con codificaciones idénticas en la población, e incluso puede interpretarse como un indicador de convergencia. Sin embargo, el fenómeno observado es diferente: individuos genotípicamente distintos representan la misma solución fenotípica debido al renombrado arbitrario de etiquetas. El problema no radica en la multiplicidad de codificaciones per se, sino en que dicha no biyectividad altera la semántica de los operadores evolutivos.

Como consecuencia, se introduce una ilusión de diversidad: el algoritmo aparenta explorar regiones distintas del espacio cuando, en realidad, está asignando recursos poblacionales a variantes equivalentes de una misma partición. En la implementación basada en DEAP, donde se emplea un mecanismo de caché para evitar la reevaluación de individuos previamente analizados, esta situación puede además ralentizar la búsqueda, ya que las representaciones redundantes no son reconocidas como tales.

En consecuencia, la redundancia estructural no solo reduce la diversidad efectiva de la población, sino que compromete la coherencia semántica de los operadores evolutivos, afectando directamente la dinámica de exploración.

A partir de este análisis surgen tres interrogantes centrales:

- ¿Cómo se cuantifica la redundancia existente actualmente?
- ¿Qué solución se propone para mitigarla?
- ¿Cómo impacta la solución propuesta sobre la performance del enfoque?

Metodología

Se analizaron las 5120 ejecuciones previamente registradas, cada una con su correspondiente frente de Pareto final. Para cada frente, se cuantificó la redundancia presente distinguiendo explícitamente entre redundancia genotípica y redundancia por permutación de etiquetas.

Sea PF el frente de Pareto final y $|PF|$ su tamaño. Donde G es el conjunto de genotipos únicos y C es el conjunto de particiones únicas (canónicas):

Redundancia genotípica (R_G):

$$R_G = 1 - \frac{|G|}{|PF|}$$

proporción de individuos en PF que poseen una representación genotípica idéntica a la de al menos otro individuo del mismo frente.

Redundancia por permutación de etiquetas (R_p):

$$R_p = 1 - \frac{|C|}{|G|}$$

proporción de individuos genotípicamente distintos que representan una misma partición fenotípica debido al renombrado arbitrario de etiquetas.

Redundancia total (R_T):

$$R_T = 1 - \frac{|C|}{|PF|} = R_G + R_p - R_G \cdot R_p$$

proporción total de individuos redundantes en PF, considerando ambos fenómenos.

Para cada ejecución se almacenaron las métricas obtenidas en un archivo .csv, a partir del cual se calcularon estadísticas descriptivas presentadas en la subsección siguiente. El conjunto completo de datos se encuentra disponible en X.

Resultados

En la Tabla 3.2.1 se expresan los datos de redundancia empírica observada en el enfoque original.

R_G	R_p	R_T		
% promedio	% promedio	% promedio	varianza	% CV
28.37	48.31	62.26	0.1692	11.3

Tabla 3.2.1: Redundancia observada en los frentes de Pareto finales del enfoque original, descompuesta en redundancia genotípica (RG), redundancia por permutación de etiquetas (RP) y redundancia total (RT).

Los resultados muestran que la redundancia total alcanza en promedio el 62.26% del tamaño del frente de Pareto final. De este valor, la mayor contribución proviene de la redundancia por permutación de etiquetas (48.31%), ampliamente superior a la redundancia genotípica clásica (28.37%). Esto confirma que el fenómeno dominante no es la convergencia poblacional tradicional, sino la redundancia estructural inducida por la representación.

Si bien este valor podría interpretarse como elevado, debe considerarse que los datos corresponden a los frentes luego de 100 generaciones, donde cierto grado de redundancia es esperable debido a convergencia. Una métrica potencialmente más informativa para evaluar el impacto dinámico del fenómeno consistiría en analizar la redundancia a lo largo de las generaciones, sin embargo, debido al costo computacional asociado a dicho análisis, el estudio se limita en este trabajo a la redundancia observada en los frentes finales.

Este hallazgo motivó la redefinición del esquema de representación y de los operadores asociados, presentada en la Sección 3.3.3. El impacto de dicha modificación sobre el desempeño del enfoque se analiza en la Sección 3.4.1.

3.3. Alteración

3.3.1. Implementación de Hypervolume (HV)

Exploración del Espacio de Soluciones

Como se precisó en la Sección 3.2.1, se optó por el indicador Hypervolume (HV) para evaluar el desempeño del algoritmo.

El rol de dicho punto es delimitar la región del espacio objetivo que será considerada en la medición. La elección de este punto es crítica, ya que debe representar un límite suficientemente amplio del espacio de soluciones posibles. Si el punto de referencia se encuentra demasiado cerca del frente de Pareto, el valor de HV puede resultar artificialmente reducido, mientras que si se encuentra demasiado lejos, puede distorsionar la comparación entre algoritmos.

Para estimar un punto de referencia adecuado para cada instancia, se realizó un estudio exploratorio del espacio de soluciones. El objetivo de este estudio fue aproximar los rangos alcanzables por cada una de las funciones objetivo en el espacio fenotípico.

Con este propósito se utilizó un motor de muestreo Monte Carlo, el cual genera soluciones aleatorias dentro del espacio de búsqueda del problema. Cada solución generada es evaluada mediante las funciones objetivo, permitiendo registrar los valores mínimos y máximos observados para cada métrica.

En particular, se generaron $M = 10^7$ soluciones aleatorias. Para cada muestra se construyó un individuo válido dentro del espacio de búsqueda, se evaluaron sus funciones objetivo y se almacenaron los valores obtenidos.

A partir de este conjunto de muestras se estimaron los extremos del espacio objetivo mediante el cálculo de los valores mínimos y máximos observados para cada función objetivo. Específicamente, si f_i denota el i -ésimo objetivo, se estimaron:

$$\min(f_i), \quad \max(f_i)$$

sobre el conjunto de soluciones generadas.

Estos valores permiten aproximar los límites del espacio objetivo y, a partir de ellos, definir un punto de referencia consistente para el cálculo de la métrica HV . Si bien este método no garantiza encontrar los extremos exactos del espacio objetivo, permite obtener una aproximación empírica suficientemente amplia del rango de valores alcanzables, lo cual resulta adecuado para la definición de un punto de referencia estable para el cálculo del HV .

Determinación del Punto de Referencia

El punto de referencia debe mantenerse fijo a lo largo de todos los experimentos, ya que el valor del indicador HV depende directamente de su ubicación. Modificarlo comprometería la consistencia de las comparaciones entre distintas configuraciones del algoritmo.

INSTANCIA	NED	SM	ICP	IN
JPetStore	1.0	-0.0532	0.7826087	10.0
AcmeAir	1.0	-0.0817	1.0	9.5
CargoTracker	1.0	0.0	1.0	9.0

Tabla 3.3.1. Extremos estimados utilizados para construir los puntos de referencia del indicador de hipervolumen (HV). A partir de estos valores se aplica posteriormente un margen adicional del 5 %.

En este trabajo, el punto de referencia se considera parte de los metadatos asociados a cada instancia del problema. En particular, se almacena en un archivo *metadata.json*, junto con otra información descriptiva de la instancia.

A partir de los rangos estimados mediante el muestreo Monte Carlo, el punto de referencia se define utilizando los extremos más desfavorables para cada función objetivo. En particular, para los objetivos de minimización (NED, ICP e IN) se utilizan los valores máximos observados, mientras que para el objetivo de maximización (SM) se utiliza el valor mínimo.

Con el fin de garantizar que el punto de referencia sea estrictamente peor que cualquier solución esperable del frente aproximado, se aplica un margen adicional del 5 % sobre dichos extremos. De este modo aumenta la probabilidad de que el punto de referencia represente una solución peor que cualquier solución alcanzable en el espacio objetivo.

La Tabla 3.3.1 resume los puntos de referencia utilizados en cada instancia experimental.

Es importante aclarar que los valores reportados en la tabla fueron estimados utilizando la representación reformulada presentada en la Sección 3.3.3. En la representación original, la redundancia por permutación de etiquetas inflaba artificialmente el espacio de búsqueda, lo que dificultaba la estimación de los extremos mediante muestreo Monte Carlo.

Con la nueva representación, el espacio de soluciones coincide directamente con el conjunto de particiones válidas, lo que permitió obtener estimaciones más estables de los rangos de las funciones objetivo. No obstante, todos los experimentos comparativos se realizaron utilizando un único punto de referencia fijo, garantizando la consistencia de las mediciones de *HV*.

Cálculo del *HV*

El indicador *HV* se calcula sobre el frente de Pareto aproximado obtenido al final de cada ejecución del algoritmo. Dado que la implementación utilizada asume que todos los objetivos son de minimización, fue necesario realizar una transformación previa sobre la métrica SM, originalmente formulada como objetivo de maximización.

En particular, los valores de SM se multiplican por -1 antes del cálculo del indicador, de modo que todos los objetivos puedan tratarse de manera uniforme como problemas de minimización.

Una vez aplicada esta transformación, el cálculo del indicador se implementó utilizando la biblioteca pymoo [11], que provee una implementación eficiente del *HV*. La implementación completa utilizada en este trabajo se encuentra disponible en el repositorio del proyecto.

El valor resultante representa el volumen del espacio objetivo dominado por el frente aproximado. En consecuencia, valores más altos de *HV* indican frentes de Pareto de mayor calidad, ya que reflejan simultáneamente mayor convergencia hacia el frente óptimo y mejor diversidad de soluciones.

Una vez validada la métrica, estamos en condiciones de avanzar al Análisis de Sensibilidad que se detalla en la sección 3.2.2.

3.3.2. Modificación de la métrica NED

Problemática

El estudio del espacio de soluciones discutido en la sección anterior permitió identificar una limitación en una de las métricas objetivo utilizadas: Non-Extreme Distribution (NED).

El objetivo de esta métrica es favorecer particiones en las que los microservicios presenten tamaños intermedios, penalizando aquellas configuraciones en las que aparecen microservicios extremadamente pequeños o excesivamente grandes.

Formalmente, NED mide la proporción de microservicios cuyo tamaño se encuentra fuera de un intervalo considerado razonable. Sea k el número de microservicios y n la cantidad de microservicios cuyo tamaño se encuentra dentro de dicho intervalo. La métrica se define como:

$$NED = 1 - \frac{n}{k}$$

Dado que el problema se formula como una optimización de minimización, valores cercanos a 0 indican particiones con tamaños más equilibrados, mientras que valores cercanos a 1 representan configuraciones con una alta presencia de microservicios extremos.

```
mondec > ea_performance.py > ...
1 import numpy as np
2 from pymoo.indicators.hv import HV
3 from mondec.config_instance import load_range_from_metadata
4
5 def hv(pareto_front):
6
7     MINS, MAXS = load_range_from_metadata()
8
9     front = np.array([ind.fitness.values for ind in pareto_front])
10
11     front[:, 1] *= -1      # negación de SM en los individuos del frente
12
13     ref_point = np.array([
14         MAXS[0],
15         -MINS[1],
16         MAXS[2],
17         MAXS[3]
18     ]) * 1.05
19
20     hv = HV(ref_point=ref_point)
21     hv_value = hv(front)
22
23     return hv_value
```

Figura 3.3.1: Implementación del procedimiento utilizado para calcular el hipervolumen (HV). El punto de referencia se construye a partir de los extremos estimados de cada objetivo y se aplica un margen adicional del 5 %, garantizando que dicho punto sea dominado por cualquier solución factible.

En su definición original, el intervalo de tamaños permitido se encontraba fijado mediante límites estáticos. Sin embargo, al aplicar esta métrica a sistemas de tamaño reducido, como JPetStore, se observó que la mayoría de las particiones generadas por el algoritmo producían microservicios cuyo tamaño quedaba fuera de dicho intervalo. Como consecuencia, la métrica tomaba frecuentemente el valor máximo $NED = 1$ para gran parte del espacio de soluciones.

Esto resulta problemático para el proceso de optimización, ya que la métrica deja de aportar información útil al algoritmo evolutivo para diferenciar entre soluciones alternativas.

Resolución

Para resolver esta limitación, se redefinieron los límites del intervalo utilizando umbrales proporcionales a N el número total de clases del sistema. En particular, se establecieron límites equivalentes al 5% y 40% del tamaño del sistema. De esta forma, la evaluación de los tamaños de los microservicios se adapta a la escala del sistema analizado, evitando la saturación de la métrica en instancias pequeñas.

La nueva métrica NED entonces se define entonces como:

$$f_1 = NED = 1 - \frac{|\{M_i \mid 0.05N < |M_i| < 0.4N\}|}{k}$$

Esta modificación permitió recuperar la capacidad de la métrica para distinguir entre diferentes configuraciones del espacio de soluciones, incrementando la presión selectiva ejercida durante el proceso de optimización.

3.3.3. Reformulación del Individuo y Operadores Asociados

Decisión de Diseño

Para eliminar la redundancia por permutación de etiquetas identificada en la Sección 3.2.3, se reemplaza la representación basada en etiquetas por una representación canónica de tipo conjunto de conjuntos. En esta codificación, el orden de las clases y de los microservicios es irrelevante. Únicamente importa qué clases conforman cada bloque.

A modo de ejemplo, la partición:

$[[0], [1, 2, 3, 4, 5, 6, 8, 9, 10, 11, 13, 19], [7], [12, 17, 20, 21, 22, 23], [14], [15], [16], [18]]$
constituye una solución válida independientemente de cualquier etiquetado interno.

El principal costo de esta decisión es que los operadores dejan de ser triviales, ya que deben preservar explícitamente la estructura de partición de las soluciones. En consecuencia, el desafío pasa a ser el diseño de operadores de mutación y cruzamiento que resulten efectivos sin introducir un costo computacional excesivo.

La ventaja de este enfoque es que cada modificación introduce un cambio estructural explícito en la organización de clases, evitando transformaciones equivalentes derivadas del renombrado arbitrario de microservicios.

Nueva Representación

Sea $C = \{0, 1, \dots, N - 1\}$ el conjunto de clases del monolito (en JPetStore: $N = 24$), una solución válida al problema de descomposición es una partición de C .

Definimos una partición como $P = \{B_1, B_2, \dots, B_k\}$:

$$B_i \subseteq C, \quad B_i \neq \phi, \quad B_i \cap B_j = \phi \text{ para } i \neq j, \quad \bigcup_{i=1}^k B_i = C;$$

Cada bloque B_i representa un microservicio y está definido únicamente por el conjunto de clases que contiene. El orden de los bloques es irrelevante, y dos soluciones son equivalentes si inducen la misma partición de C .

Bajo esta codificación, el espacio de búsqueda coincide con el conjunto de todas las particiones posibles de C , cuyo tamaño es el número de Bell, considerablemente menor que el tamaño inducido por la representación anterior.

Nuevo Operador de Mutación

Dada una partición $P = \{B_1, B_2, \dots, B_k\}$, el operador de mutación combina dos mecanismos complementarios, inicialmente cada uno seleccionado con probabilidad 0.5:

A. Operador de transferencia de clase (*transfer*):

1. Seleccionar una clase $c \in C$ al azar.
2. Sea $B_s \in P$ tal que $c \in B_s$.
3. Seleccionar un bloque destino $B_t \in P \setminus B_s$ o bien crear un nuevo bloque $B_t = \phi$.
4. Construir:
 - $B_s' = B_s \setminus \{c\}$
 - $B_t' = B_t \cup \{c\}$
5. Reemplazar en P los bloques B_s y B_t por B_s' y B_t' respectivamente.
6. Si $B_s = \phi$, eliminarlo de la partición.

B. Operador de fusión de bloques (*merge*):

1. Seleccionar dos bloques distintos $B_s, B_t \in P$ al azar.
2. Construir $B' = B_s \cup B_t$.
3. Reemplazar en P los bloques B_s y B_t por B' .

Ambos operadores preservan la validez de la partición sin necesidad de reparación. El operador de transferencia introduce granularidad fina, moviendo una clase entre microservicios o aislándola en uno nuevo. El operador de fusión introduce cambios estructurales de mayor magnitud, consolidando dos microservicios en uno.

La combinación equiprobable de ambos mecanismos mitiga el sesgo sistemático hacia la fragmentación que presentaría el operador de transferencia aplicado de forma exclusiva sumado al operador de cruzamiento, manteniendo neutro el valor esperado de k a lo largo de las generaciones.

Nuevo Operador de Cruzamiento

La estrategia elegida se denomina cruzamiento por intersección de particiones.

Dados dos padres:

- $P_A = \{A_1, A_2, \dots, A_j\}$ con $j \in [1, N]$
- $P_B = \{B_1, B_2, \dots, B_k\}$ con $k \in [1, N]$

Se construye el hijo H de la siguiente manera:

1. Inicializar $H = \emptyset$ (hijo) y $U = C$ (conjunto de clases aún no asignadas)
2. Para cada $A_i \in P_A$ y para cada $B_j \in P_B$:
 - Calcular $I_{ij} = A_i \cap B_j$
 - Si $I_{ij} \neq \emptyset \rightarrow$ agregar I_{ij} a H y actualizar U tal que $U = C \setminus \cup I_{ij}$
3. Si $U \neq \emptyset \rightarrow$ agregar U como bloque singleton (no debería ocurrir si P_A y P_B son válidas)

Este operador tiene la ventaja de que las clases que aparecen juntas en ambos padres tienden a permanecer juntas, y cuando los padres difieren, la diferencia se traduce en una reorganización concreta. No mezcla segmentos arbitrarios del genotipo, sino que opera directamente sobre bloques con significado modular.

Por otro lado, una desventaja del operador es su tendencia natural a la fragmentación (el número de microservicios de H siempre es igual o mayor al de sus padres). Este efecto se compensa mediante la mutación, que tiende a la fusión de microservicios, y por la presión selectiva, que tiende a descartar soluciones con distribuciones extremas y alto acoplamiento.

Al igual que el operador de mutación, este mecanismo tampoco requiere de reparación posterior: si los padres P_A y P_B son particiones válidas de C , el hijo H también lo será.

Función de Evaluación

La función de evaluación no requiere modificaciones sustanciales, ya que opera conceptualmente sobre particiones del conjunto de clases. En la implementación original, el individuo era traducido desde su representación basada en etiquetas hacia una estructura equivalente a una partición antes de ser evaluado.

Con la nueva representación canónica, dicha traducción se simplifica. Únicamente se reemplaza el mapeo previo por una función que transforma la estructura en el formato requerido por el evaluador.

Implementación

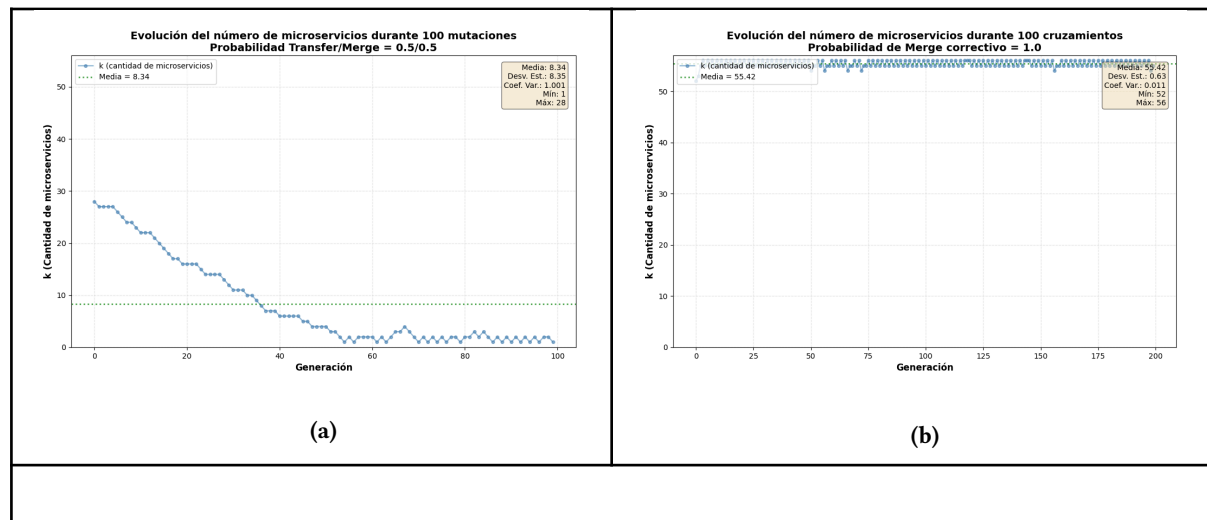
En lugar de usar la representación anterior, que consistía en una lista de enteros, se definió la clase `Individual`, que modela explícitamente una partición como un conjunto inmutable de bloques (`frozenset` de `frozenset`).

Se redefinieron como funciones la mutación y el cruzamiento, así como la inicialización, que genera una partición válida distribuyendo las clases en k bloques aleatorios, y la traducción, que convierte la partición en el formato requerido por las métricas (diccionario de microservicios). También se definieron a nivel de métodos de la clase `Individual` la copia, la comparación con otro individuo, la conversión a string, como requisitos internos de DEAP.

Finalmente, se registraron estos cambios en la configuración del algoritmo. El reemplazo fue relativamente directo porque DEAP desacopla representación y operadores, lo que permitió cambiar el modelo interno sin modificar la lógica evolutiva general.

Los operadores se evaluaron inicialmente de forma aislada para analizar su comportamiento estructural, en particular su efecto sobre la evolución de la cantidad de microservicios k en los individuos. En términos generales, la mutación tiende a reducir la fragmentación de la partición, disminuyendo el valor promedio de k en la población, mientras que el cruzamiento tiende a incrementarla.

Para la instancia de `CargoTracking` ($N=56$) se analizó el efecto de aplicar 100 mutaciones y cruzamientos sucesivos sobre un mismo individuo inicial, obteniéndose los siguientes resultados:



En conjunto, ambos operadores generan una dinámica de balance: el cruzamiento favorece la fragmentación al aislar estructuras distintas, mientras que la mutación tiende a agrupar bloques.

En síntesis, la reformulación introduce una representación biyectiva entre genotipo y fenotipo y redefine los operadores para que actúen directamente sobre el espacio de particiones válidas, incorporando dinámicas de exploración más acordes al contexto del problema. En la Sección 3.4.1 se evalúa experimentalmente el impacto de estos cambios sobre la calidad de las soluciones obtenidas y el desempeño general del enfoque.

3.4. Evaluación

3.4.1. Impacto de la Nueva Representación

Recordemos que en la Sección 3.2.3 se advirtió el problema de la redundancia por permutación de etiquetas, señalada por su potencial para entorpecer la dinámica de búsqueda del algoritmo al introducir falsa diversidad en la población. En la Sección 3.3.3 se propuso resolver este problema reemplazando la representación original por una representación canónica basada en conjuntos de conjuntos, lo que implicó necesariamente rediseñar los operadores de mutación y cruzamiento, y por ende, la dinámica de exploración del espacio de soluciones.

A continuación se evalúa el impacto de estos cambios sobre el desempeño del algoritmo, considerando principalmente dos aspectos: la calidad de los frentes obtenidos, medida mediante hipervolumen (HV), y la robustez del enfoque, medida a partir de la variabilidad del HV entre ejecuciones.

Eliminación de redundancia por permutación de etiquetas

Una consecuencia esperable de reemplazar la representación original por una representación canónica es la eliminación de la redundancia por permutación de etiquetas. A continuación se cuantifica, utilizando el mismo procedimiento descrito en la Sección 3.2.3, la redundancia presente en los frentes de Pareto finales.

R_G	R_P	R_T		
% promedio	% promedio	% promedio	varianza	% CV
0.42	0.00	0.42	0.1581	37.64

Tabla 3.4.1: Redundancia observada en los frentes de Pareto finales bajo la nueva representación, descompuesta en redundancia genotípica (R_G), redundancia por permutación de etiquetas (R_P) y redundancia total (R_T).

La redundancia por permutación de etiquetas desaparece completamente bajo la nueva representación y, en consecuencia, la redundancia total se reduce respecto al enfoque original. Esto implica que la redundancia restante corresponde únicamente a similitudes estructurales reales entre soluciones, y no a duplicaciones introducidas por la representación.

Para evaluar si esta reducción de redundancia se traduce en mejoras en la calidad o diversidad de las soluciones obtenidas, resulta necesario analizar la evolución del indicador Hypervolume a lo largo del proceso evolutivo, lo cual se presenta en la sección siguiente.

Calidad del frente a lo largo del tiempo

Para evaluar el impacto de la nueva representación y de los operadores de variación definidos sobre ella, un análisis solo estructural resulta insuficiente. Una forma más informativa consiste en analizar cómo evoluciona la calidad del frente de Pareto a lo largo del proceso evolutivo.

Para ello, se estudió la evolución del HV a lo largo de las generaciones. La Figura 3.4.1 muestra la evolución del hipervolumen promedio a lo largo de las generaciones para ambos enfoques.

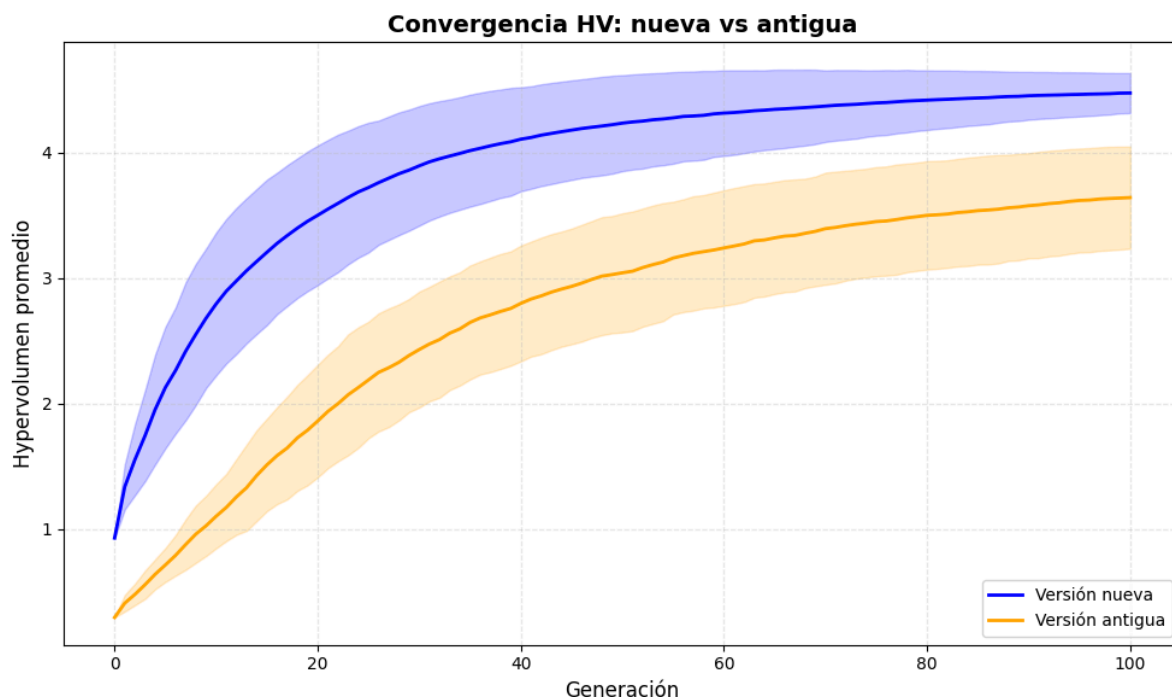


Figura 3.4.1: Evolución del hipervolumen promedio (HV) a lo largo de 100 generaciones para el enfoque original (en naranja) y el nuevo enfoque (en azul), promediado sobre 30 ejecuciones independientes. Las bandas sombreadas indican la variabilidad entre ejecuciones.

La metodología experimental fue la siguiente:

- Se utilizó la instancia JPetStore ($N = 24$ clases).
- Se generaron 30 configuraciones de hiperparámetros mediante muestreo de Saltelli.
- La semilla se fijó en 42, de modo que cada configuración produjera trayectorias comparables entre enfoques.
- Para cada ejecución se registró el valor de HV en cada una de las 100 generaciones.
- El procedimiento se repitió tanto para el enfoque original como para el enfoque con la nueva representación, utilizando exactamente las mismas configuraciones y en el mismo orden.
- Para el cálculo del HV se utilizó el mismo punto de referencia en todos los casos, garantizando la comparabilidad de los resultados.

A partir de los datos recopilados, se calculó para cada generación el *HV* promedio y su variabilidad entre ejecuciones. Luego se graficaron ambas curvas, donde el eje x representa la generación y el eje y el hipervolumen promedio alcanzado. Este análisis permite observar no solo la calidad final de los frentes obtenidos, sino también la velocidad de convergencia del algoritmo.

Cuantificación de la Mejora

Para cuantificar la diferencia observada en la figura anterior se utilizó la métrica AUC_{HV} (*Area Under the Curve for Hypervolume*), definida como la suma del HV a lo largo de las generaciones:

$$AUC_{HV} = \sum_{g=1}^{100} HV_g$$

Esta métrica captura simultáneamente la calidad del frente y la velocidad de convergencia, ya que penaliza enfoques que alcanzan buenos frentes únicamente en generaciones tardías. Los resultados obtenidos se reportan en la Tabla 3.4.2.

Enfoque	AUC_{HV}	Mejora	σAUC_{HV}
Original	270.09	↑ 44.56 %	38.9659
Nuevo	390.46		35.2264

Tabla 3.4.2: Comparación del área bajo la curva entre el enfoque original y la nueva representación. La nueva representación obtiene un área bajo la curva promedio 44.56 % mayor y una ligera reducción en la desviación estándar.

La nueva representación alcanza un AUC_{HV} promedio un 44.5 % mayor que el enfoque original. Esto indica que, en promedio, el algoritmo no solo obtiene frentes de mejor calidad, sino que además los alcanza considerablemente más rápido a lo largo del proceso evolutivo. Adicionalmente, se observa una ligera reducción en la desviación estándar, lo que sugiere una mayor estabilidad del enfoque basado en la nueva representación.

Este resultado es consistente con la dinámica observada en la Figura 3.4.1, donde la curva correspondiente a la nueva representación domina sistemáticamente a la del enfoque original a lo largo de las generaciones.

Calidad del output

Para complementar el análisis anterior, también se comparó el hipervolumen obtenido en la última generación. Los resultados se presentan en la Tabla 3.4.3 y se calcularon considerando las 5120 ejecuciones realizadas para el análisis de sensibilidad.

La nueva representación produce frentes finales con un aumento promedio del 35.37% en el HV final, además de mostrar una reducción significativa en la variabilidad entre ejecuciones, de aproximadamente 60.93%. En conjunto, estos resultados indican que la nueva representación no solo mejora la calidad final de las soluciones obtenidas, sino que también incrementa la robustez del algoritmo frente a distintas ejecuciones.

Enfoque	HV_f	Mejora	σHV_f
Original	3.2849	↑ 35.37 %	0.3679
Nuevo	4.4473		0.2273

Tabla 3.4.3: Comparación del hipervolumen final (HV_f) entre la representación original y la nueva representación.

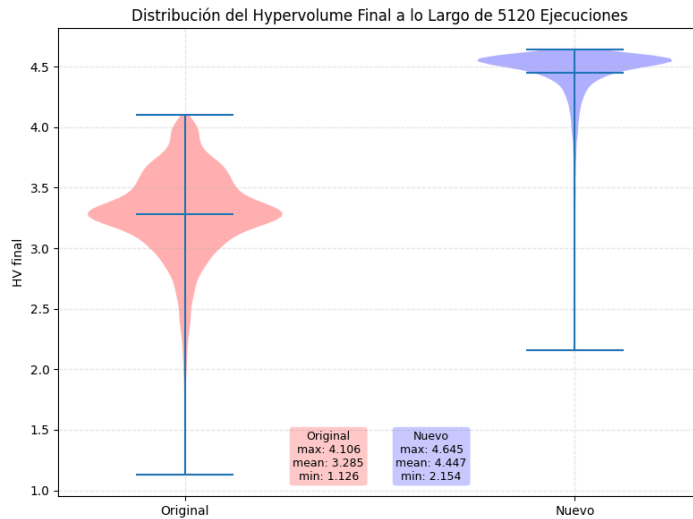


Figura 3.4.2: Diagrama de violín de la distribución del hipervolumen final en 5120 ejecuciones para la representación original (en rojo) y la nueva representación (en azul).

La Figura 3.4.2 presenta además un diagrama de violín de la distribución del HV final. En él se observa que los valores obtenidos con la nueva representación dominan sistemáticamente a los del enfoque original, mostrando además una distribución más concentrada.

3.4.2. Nuevo Análisis de Sensibilidad

Motivación

Dado que la redefinición del individuo y de los operadores de variación introduce cambios sustanciales en la dinámica de búsqueda del algoritmo evolutivo, resulta necesario repetir el análisis de sensibilidad global. Esto permite evaluar si la influencia relativa de los hiperparámetros se mantiene o si el nuevo diseño modifica el comportamiento del algoritmo.

Metodología

El análisis se realizó siguiendo exactamente la misma metodología descrita en la Sección 3.2.2. Esto garantiza que los resultados obtenidos sean directamente comparables con los del algoritmo original.

Resultados

La Tabla 3.4.4 presenta los índices de Sobol obtenidos para los tres hiperparámetros considerados.

Parámetro	$S1$	ST	$IC(S1)$	$IC(ST)$
μ	0.5268	0.7328	0.1096	0.1366
λ/μ	0.1999	0.4322	0.1043	0.1132
p_mut	0.0838	0.2876	0.0501	0.0936

Tabla 3.4.4: Nuevos índices de sensibilidad de Sobol para los hiperparámetros del algoritmo evolutivo, calculados sobre el indicador Hypervolume (HV). Se reportan los índices de primer orden ($S1$), los índices totales (ST) y los intervalos de confianza (IC) asociados.

Interpretación

Los resultados obtenidos muestran que el tamaño poblacional (μ) continúa siendo el hiperparámetro con mayor influencia sobre el desempeño del algoritmo. En este nuevo diseño, su índice de primer orden alcanza $S1 = 0.5268$, considerablemente superior al observado en el análisis previo ($S1 = 0.3617$). Asimismo, su índice total aumenta hasta $ST = 0.7328$, lo que indica que el tamaño de la población explica una proporción aún mayor de la variabilidad observada en el indicador HV.

Por su parte, la relación entre descendientes y población (λ/μ) mantiene una influencia moderada, con $S1 = 0.1999$ y $ST = 0.4322$, valores comparables a los obtenidos en el análisis anterior. Esto sugiere que la proporción de descendientes continúa desempeñando un rol relevante en la dinámica de exploración del algoritmo, particularmente a través de su interacción con el tamaño poblacional.

La principal diferencia respecto al análisis anterior se observa en la probabilidad de mutación, cuyo efecto directo disminuye de $S1 = 0.2358$ a $S1 = 0.0838$. De forma similar, su índice total se reduce de $ST = 0.5016$ a $ST = 0.2876$. Este resultado indica que, bajo la nueva representación del individuo y los operadores de variación, la probabilidad de mutación tiene una influencia considerablemente menor sobre la variabilidad del desempeño del algoritmo.

En conjunto, estos resultados sugieren que el nuevo diseño del algoritmo ha desplazado el peso de la dinámica de búsqueda hacia el tamaño poblacional, mientras que la influencia relativa de los operadores de variación se ha reducido. Esto podría interpretarse como una señal de que la nueva representación produce operadores más robustos o menos sensibles a la parametrización de mutación.

Finalmente, la comparación entre los índices de primer orden y de orden total indica que siguen existiendo interacciones relevantes entre los hiperparámetros, aunque la magnitud relativa de estas interacciones se mantiene similar a la observada en el análisis original.

La Figura 3.4.1 muestra visualmente el incremento en la contribución del tamaño poblacional y la reducción en la influencia de la probabilidad de mutación respecto al diseño original.

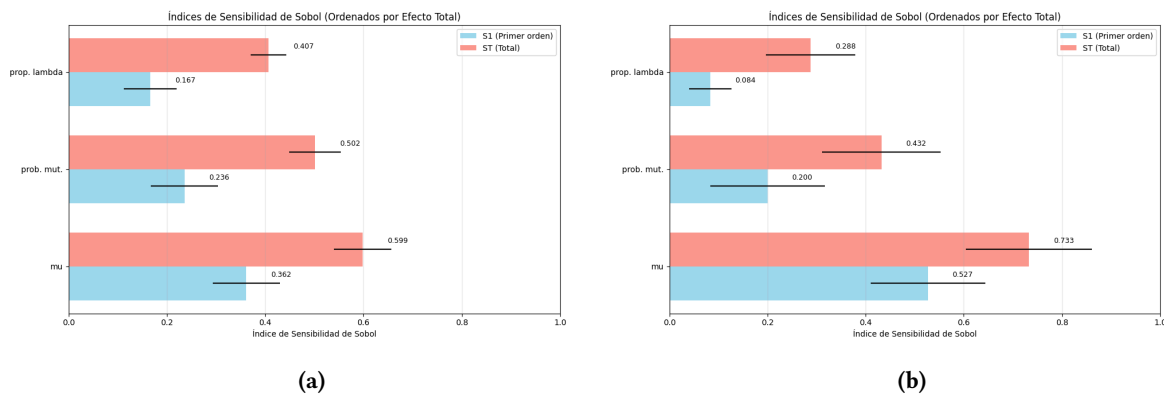


Figura 3.4.1: Índices de sensibilidad de Sobol para los hiperparámetros evaluados, ordenados según su efecto total. Las barras azules representan los índices de primer orden (S1) y las rojas los índices de orden total (ST). Las subfiguras (a) y (b) muestran los resultados para el enfoque original y el nuevo enfoque, respectivamente.

Otros Hallazgos

Al igual que en la Sección 3.4.5, se analizaron las configuraciones correspondientes al 1 % superior e inferior de ejecuciones según su valor final de Hypervolume (HV). Las estadísticas descriptivas de ambos grupos se presentan en la Tabla 3.4.5. La figura 3.4.2 presenta una comparación final entre las ejecuciones en ambos extremos para los dos enfoques.

Las configuraciones de mayor rendimiento presentan valores de HV altamente concentrados en un intervalo muy estrecho ($CV \approx 0.0013$), ligeramente más estrecho que el top 1% del enfoque anterior ($CV \approx 0.0046$), lo que indica que bajo dichas condiciones el algoritmo converge de manera consistente hacia frentes de Pareto de calidad muy similar. En contraste, las configuraciones del grupo inferior presentan una variabilidad considerablemente mayor ($CV \approx 0.0871$), lo que sugiere que esas regiones del espacio de hiperparámetros producen resultados menos estables, aunque más estables que el grupo inferior del enfoque anterior ($CV \approx 0.137$).

En términos de los hiperparámetros, se observa nuevamente que las configuraciones de mejor desempeño se asocian con poblaciones más grandes. El tamaño poblacional promedio del grupo superior alcanza $\mu \approx 896$, mientras que en el grupo inferior es de aproximadamente $\mu \approx 247$, lo que refuerza el resultado obtenido en el análisis de sensibilidad, donde μ aparece como el hiperparámetro con mayor influencia sobre el desempeño del algoritmo. Esto condice con el análisis del enfoque original.

De forma similar, las configuraciones exitosas presentan una mayor cantidad de descendientes generados por generación ($\lambda \approx 2376$ frente a $\lambda \approx 334$), lo que sugiere que una mayor producción de individuos favorece la exploración efectiva del espacio de soluciones, comportamiento que también se observaba en el enfoque original.

En cuanto a los operadores evolutivos, se observa una diferencia interesante respecto al análisis del algoritmo original. Mientras que anteriormente las configuraciones de mejor desempeño tendían a presentar probabilidades de mutación muy elevadas, en el nuevo diseño los mejores resultados se obtienen con valores más moderados de mutación ($p_{mut} \approx 0.47$). Este comportamiento es

	top 1%					bottom 1%				
	mu	lambda	p_mut	p_cx	HV	mu	lambda	p_mut	p_cx	HV
count	52	52	52	52	52	52	52	52	52	52
mean	895.90	2376.38	0.4721	0.5278	4.6214	247.00	333.75	0.0668	0.9331	3.1395
std	76.71	341.00	0.2110	0.2110	0.0060	268.20	355.68	0.0893	0.0893	0.2736
cv	0.0856	0.1435	0.4469	0.3998	0.0013	1.0858	1.0657	1.3368	0.0957	0.0871
min	698.00	1685.00	0.0754	0.1687	4.6148	50.00	57.00	0.0005	0.6628	2.1544
max	999.00	2984.00	0.8312	0.9245	4.6451	992.00	1611.00	0.3371	0.9994	3.4368

Tabla 3.4.5: Estadísticas descriptivas de los hiperparámetros y del indicador Hypervolume (HV) para las configuraciones correspondientes al 1% superior y al 1% inferior de ejecuciones del nuevo enfoque. Se reportan el número de observaciones (count), media (mean), desvío estándar (std), coeficiente de variación (cv), y los valores mínimo y máximo observados.

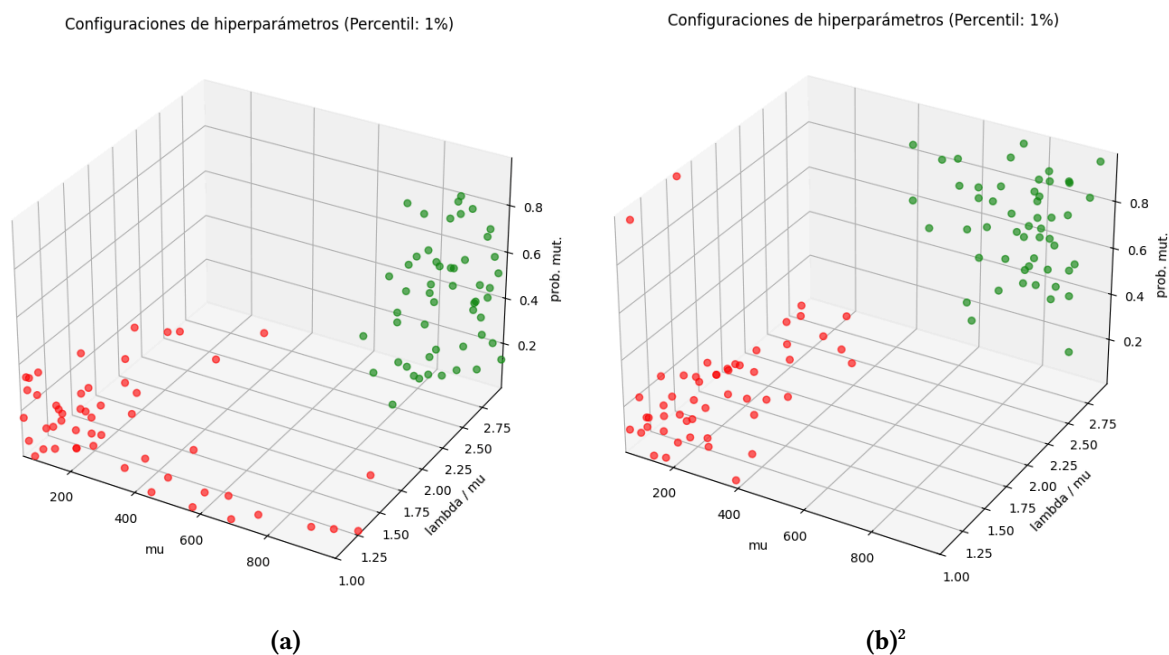


Figura 3.4.2. Configuraciones de hiperparámetros correspondientes a los extremos de desempeño del algoritmo. Los puntos verdes representan el 1 % de configuraciones con mayor hipervolumen final (HVf), mientras que los puntos rojos corresponden al 1 % con menor HVf. (a) Representación original. (b) Nueva representación.

consistente con el nuevo análisis de sensibilidad, que mostró una reducción significativa en la influencia global de este hiperparámetro.

Asimismo, el valor promedio de HV alcanzado por las configuraciones de mejor desempeño ($HV \approx 4.62$) es superior al observado en el algoritmo original ($HV \approx 4.05$). Esto sugiere que las modificaciones introducidas en la representación del individuo y en los operadores de variación contribuyen efectivamente a mejorar la calidad de los frentes de Pareto obtenidos.

La mejora resulta aún más marcada en el grupo inferior, cuyo promedio de $HV \approx 3.14$ es sustancialmente mayor que el observado en el enfoque anterior ($HV \approx 1.85$).

En conjunto, estos resultados refuerzan la conclusión de que el nuevo diseño del algoritmo permite alcanzar frentes de Pareto de mayor calidad de forma más consistente y robusta.

² La animación que muestra la oscilación del 1% al 50% de los extremos de HV está disponible en: [nuevo.gif](#)

4.1. Conclusión

Este proyecto tomó como punto de partida un enfoque evolutivo multiobjetivo existente para la descomposición de sistemas monolíticos en microservicios, y aplicó de forma iterativa un ciclo de análisis, intervención y evaluación con el objetivo de medir y mejorar su desempeño.

Los aportes principales pueden organizarse en tres líneas.

En primer lugar, se introdujo el indicador Hypervolume (HV) como métrica cuantitativa de calidad, habilitando comparaciones objetivas entre configuraciones del algoritmo que hasta entonces no eran posibles.

En segundo lugar, se identificó y cuantificó la redundancia por permutación de etiquetas presente en la representación original, y se propuso una representación canónica basada en conjuntos de conjuntos junto con operadores de variación rediseñados para operar directamente sobre el espacio de particiones válidas. Esta modificación eliminó por completo dicha redundancia y produjo mejoras sustanciales y estadísticamente significativas en la calidad de los frentes de Pareto obtenidos: un incremento del 35.37% en el HV final, con una reducción del 61% en la variabilidad entre ejecuciones.

En tercer lugar, el análisis de sensibilidad global permitió caracterizar la influencia relativa de los hiperparámetros en ambas versiones del algoritmo, revelando que el tamaño poblacional constituye el principal determinante del desempeño y que la nueva representación produce operadores más robustos y menos sensibles a la parametrización de mutación.

En conjunto, puede afirmarse que el objetivo central del proyecto fue cumplido: el enfoque resultante es más eficiente, más estable y capaz de alcanzar soluciones de mayor calidad que el original.

No obstante, corresponde señalar algunas limitaciones que acotan el alcance de estas conclusiones. La comparación experimental se realizó únicamente sobre la instancia JPetStore, por lo que la generalización de los resultados a otras instancias requiere validación adicional. Asimismo, los casos de estudio empleados son de tamaño reducido respecto a sistemas industriales reales, lo que limita la extrapolación directa de los resultados a contextos productivos. Estas consideraciones se retoman en la sección siguiente, donde se discuten posibles líneas de trabajo futuro.

4.2. Trabajo Futuro

El trabajo realizado abre diversas líneas de investigación que podrían profundizar y extender los resultados obtenidos.

Una limitación central del estudio es que la comparación experimental entre la representación original y la nueva se realizó casi exclusivamente sobre JPetStore ($N = 24$). Validar el comportamiento del enfoque propuesto sobre instancias de mayor tamaño o con características estructurales distintas permitiría evaluar la generalización del método y su escalabilidad frente a sistemas más cercanos a entornos industriales reales.

En segundo lugar, los resultados obtenidos combinan el efecto del cambio de representación con el rediseño de los operadores de variación, sin que sea posible aislar la contribución de cada componente por separado. Un experimento que evalúe cada modificación de forma independiente permitiría comprender con mayor precisión qué aspectos del nuevo diseño son responsables de las mejoras observadas.

Por otro lado, el análisis de calidad de los frentes de Pareto se basó exclusivamente en el indicador Hypervolume (HV). Construir una aproximación al frente de Pareto ideal, por ejemplo, mediante la unión de los mejores frentes obtenidos, permitiría calcular métricas complementarias como GD, IGD+ o el indicador Epsilon (ϵ), que capturan propiedades distintas del frente y ofrecerían una evaluación más completa de la calidad de las soluciones.

Otra limitación práctica del trabajo se relaciona con el uso del framework DEAP para la implementación del algoritmo. Si bien esta herramienta resulta especialmente útil para el prototipado rápido de algoritmos evolutivos, su arquitectura introduce ciertas restricciones al momento de explorar diseños algorítmicos más específicos o no convencionales. En particular, algunos aspectos de reproducibilidad y la incorporación de representaciones y operadores altamente personalizados pueden requerir adaptaciones adicionales dentro del framework. En este sentido, una posible línea de trabajo futuro consiste en desarrollar una implementación más especializada que permita un control más fino sobre estos componentes y facilite la experimentación con variantes del algoritmo.

Finalmente, podrían explorarse comparaciones con otros algoritmos evolutivos multiobjetivo como NSGA-II o MOEA/D, así como la incorporación de nuevas métricas arquitectónicas que amplíen el espacio de objetivos considerados.

4.3. Reflexiones

Relación con la Carrera

Este proyecto representó una oportunidad para integrar distintos conocimientos adquiridos a lo largo de la carrera de Ingeniería de Sistemas. Algunas materias fueron especialmente relevantes:

- **Introducción a la Computación Evolutiva** proporcionó la base conceptual del enfoque evolutivo, incluyendo la representación de soluciones, los operadores de selección, cruzamiento y mutación, y las buenas prácticas en el diseño de algoritmos evolutivos.
- **Análisis y Diseño de Algoritmos (AyDA) I y II** aportó herramientas para el modelado algorítmico de problemas, el análisis de complejidad y el diseño de estrategias heurísticas. Además, permitió reconocer similitudes con problemas clásicos de optimización, como *bin-packing*, y presentó familias de técnicas alternativas para atacar este tipo de problemas.
- **Investigación Operativa** reforzó la comprensión de optimización en sentido amplio, con ejemplos en múltiples dominios aplicados, instruyendo sobre modelos, variables, restricciones, técnicas de resolución y análisis de trade-offs.
- **Metodologías de Desarrollo de Software** contribuyó a estructurar el proceso de trabajo a través de prácticas como la documentación, el desarrollo iterativo, la trazabilidad del mismo y el uso de herramientas como GitHub para la gestión del proyecto.
- **Diseño de Software** aportó criterios para la organización del código, la modularización de los componentes y la implementación de soluciones reproducibles, aspectos especialmente importantes en un proyecto con una fuerte componente experimental.

En conjunto, estas materias proporcionaron las herramientas conceptuales y metodológicas necesarias para abordar el problema desde una perspectiva interdisciplinaria, combinando fundamentos de optimización, diseño algorítmico y arquitectura de software.

Aprendizajes No Técnicos

Más allá de los aspectos técnicos, el proyecto dejó aprendizajes igualmente valiosos sobre el propio proceso de investigación.

Uno de los más significativos fue aprender a tolerar la incertidumbre inherente al trabajo investigativo. En muchas ocasiones no resultaba claro cuál era el camino correcto, los resultados no tenían un sentido inmediato o era necesario revisar supuestos que parecían sólidos. La investigación no siempre avanza de forma lineal ni predecible, y desarrollar tolerancia a esa ambigüedad fue un aprendizaje que se fue construyendo a lo largo de todo el proceso.

El proyecto fue también un ejercicio sostenido de gestión del tiempo, de toma de decisiones y manejo de la frustración frente a obstáculos que no siempre tenían una solución inmediata. Asimismo, implicó aprender a trabajar sobre un proyecto existente: comprender el trabajo previo de otras personas e identificar dónde era posible aportar valor.

Otro aprendizaje relevante fue observar de cerca el rigor que implica realizar un aporte dentro del ámbito científico, algo que se hizo especialmente visible durante el proceso de postulación del trabajo a *Genetic and Evolutionary Computation Conference (GECCO)*.

No quería finalizar mi trayectoria como estudiante sin el difícil souvenir de una identidad profesional. ¿Qué hacer con este título? El deseo de autonomía pedía que tomara esa decisión de antemano. En esa búsqueda decidí aprovechar las optativas como lo que son: una oportunidad para explorar. Fue por eso que, de las cinco optativas requeridas para recibirme, cursé doce.

Una de ellas, Introducción a la Computación Evolutiva, dictada por la Dra. Virginia Yannibelli, llamó fuertemente mi atención. Se trataba de una rama de la inteligencia artificial de la que jamás había escuchado antes, cuyo concepto central consiste en utilizar mecanismos inspirados en la evolución biológica —responsables de dar forma a los sistemas vivos durante millones de años— para abordar una gran variedad de problemas en distintos campos.

Me resultó fascinante. Recuerdo especialmente un ejemplo presentado en clase: el uso de técnicas evolutivas para asistir en el diseño de una antena satelital más eficiente que cualquier diseño previo realizado por humanos. Creo que en ese momento empecé realmente a ver la tecnología ya no como protagonista u objeto central de estudio, sino como un soporte que conecta y sostiene el avance de otras disciplinas no menos fascinantes.

Con mi atención captada por esta área, sumado a un interés incipiente por la tecnología detrás de la exploración espacial, y todavía con muchísimas dudas, me convencí de que mi intercambio en Toulouse, Francia, gracias al programa ARFITEC —experiencia por la que nunca voy a dejar de estar agradecida a mi universidad— iba a proporcionarme la respuesta que buscaba.

Por una de esas casualidades de la vida, Toulouse es considerada una capital aeroespacial mundial y, mientras yo buscaba desesperadamente una identidad profesional, encontré algo que se parecía bastante a una: el cruce de disciplinas en la complejidad. Descubrí que me interesaba la computación evolutiva por la misma razón que me atraía el campo aeroespacial: la oportunidad de trabajar junto a personas de trasfondos muy distintos, altamente capaces, intentando comprender y construir sistemas complejos que emergen de muchas partes pequeñas bien diseñadas y que buscan, en conjunto, correr los límites de lo humano.

Al volver, me alegré de poder integrarme a este proyecto que, a mi parecer, refleja justamente una intersección entre dos mundos. Por un lado, el de los procesos que nos inspiran: en este caso la evolución que emerge de la biología, pero que bien podrían ser también el espacio, la mente humana o cualquier sistema complejo cuyo comportamiento todavía albergue algo de misterio. Por otro lado, el mundo de sistemas, que es donde fui formada: la disciplina de construir software con rigor, de diseñar arquitecturas que resistan el tiempo y de tratar cada detalle como parte de un sistema mayor. Es ese enfoque el que permite tomar ideas que parecen lejanas o abstractas y convertirlas en algo que realmente puede existir y funcionar.

Mi búsqueda fue —y sigue siendo— bastante caótica. Todavía no sé exactamente qué quiero hacer con este título. De hecho, hoy estoy trabajando en un prototipo de detección de descarrilamientos ferroviarios, algo que poco tiene que ver con la evolución, ni con la exploración espacial, ni siquiera con microservicios.

Ya cerrando este ciclo, pienso que quizás ese sea justamente el sentido de esta formación. Más que una respuesta definitiva, este título funciona para mí como una herramienta para navegar el mundo y ver qué tiene —y qué tengo— para ofrecer. No solo me llevo teoría y habilidades técnicas: me llevo también una identidad viva que fue madurando durante años en un ambiente que me dio algunos de los momentos más felices y constructivos de mi vida. Y eso es algo para agradecer.

Agradecimientos

En primer lugar, quiero agradecer a mis directores, el Dr. Guillermo Rodríguez y Ana Martínez Saucedo, por haberme dado la oportunidad de incorporarme a este proyecto y por acompañarme a lo largo de todo el proceso. Su disposición y su guía fueron fundamentales para poder avanzar. Trabajar junto a ellos fue una experiencia formativa y humana muy valiosa.

A la Dra. Virginia Yannibelli, cuya materia despertó mi interés por la computación evolutiva y cuya predisposición a colaborar durante el desarrollo del trabajo fue siempre generosa y oportuna.

A mi familia y a mis amigos, por acompañarme con amor y paciencia en todo este proceso, tanto en las alegrías —que abundaron— como en los momentos arduos.

Finalmente, a la Universidad Nacional del Centro de la Provincia de Buenos Aires, por la formación, la contención y las oportunidades que me brindó durante mi tiempo como estudiante.

Referencias

- [1] A. Martínez Saucedo, G. Rodríguez, F. Gomes Rocha, R. Pereira dos Santos, "Migration of monolithic systems to microservices: A systematic mapping study," *Information and Software Technology*, vol. 177, 2025.
- [2] K. Deb, H. Jain, "An Evolutionary Many-Objective Optimization Algorithm Using Reference-Point-Based Nondominated Sorting Approach, Part I: Solving Problems With Box Constraints," *IEEE Transactions on Evolutionary Computation*, vol. 18, no. 4, pp. 577-601, 2014.
- [3] A. Martínez Saucedo, G. Rodríguez, F. Gomes Rocha, R. Pereira dos Santos, "Migration of monolithic systems to microservices: A systematic mapping study," *Information and Software Technology*, vol. 177, 2025.
- [4] F.-A. Fortin, F.-M. De Rainville, M.-A. Gardner, M. Parizeau, and C. Gagné, "DEAP: Evolutionary algorithms made easy," *Journal of Machine Learning Research*, vol. 13, pp. 2171–2175, Jul. 2012.
- [5] MyBatis, "JPetStore 6," Sample Java e-commerce application. [Online]. Available: <https://github.com/mybatis/jpetstore-6>
- [6] V. Vernon, "CargoTracker: A Domain-Driven Design sample application," Sample logistics and cargo tracking system implementing DDD. [Online]. Available: <https://github.com/citerus/dddsample-core>
- [7] IBM, "AcmeAir: Microservices benchmark application," Cloud-native airline booking system for benchmarking microservice architectures. [Online]. Available: <https://github.com/acmeair/acmeair>
- [8] Performance Evaluation Metrics for Multi-Objective Evolutionary Algorithms, Applied Sciences (MDPI), 2021.
- [9] [\(PDF\) Performance metrics in multi-objective optimization](#)
- [10] J. Herman and W. Usher, "SALib: An open-source Python library for sensitivity analysis," *Journal of Open Source Software*, vol. 2, no. 9, p. 97, 2017, doi: 10.21105/joss.00097.
- [11] J. Blank and K. Deb, "pymoo: Multi-objective Optimization in Python," *IEEE Access*, vol. 8, pp. 89497–89509, 2020, doi: 10.1109/ACCESS.2020.2990567.
- [12]